Ingénierie de Données & Business Intelligence

Licence Analytique des Données

Indication

> **Mexora** est une plateforme e-commerce marocaine fondée à Tanger. Elle commercialise des produits électroniques, de mode et d'alimentation. Cette entreprise fictive traverse tous les chapitres du cours.

NB : Installer le SGBD Postgres et L'éditeur DBeaver

📋 Plan du cours

	Chapitre
1	Fondements de l'Informatique Décisionnelle
2	Modélisation Dimensionnelle
3	Processus ETL / ELT
4	Data Warehouse avec PostgreSQL
5	Architectures Modernes de la Donnée
6	Visualisation & Restitution

	Chapitre
7	Visualisation avec Metabase
8	Gouvernance & Qualité des Données
9	Perspectives : IA & Futur de la Donnée

Chapitre 1 — Fondements de l'Informatique Décisionnelle | Foundations of Business Intelligence

1.1 Pourquoi la Business Intelligence existe-t-elle ?

Le problème de départ

Mexora enregistre chaque jour des milliers de commandes, paiements, retours et livraisons dans sa base de données MySQL, depuis son siège à Tanger. Ce système fonctionne parfaitement pour ce qu'il est conçu à faire : gérer les transactions courantes en temps réel.

Un jour, le Directeur Général pose une question apparemment simple :

> *"Quel est notre chiffre d'affaires par région et par catégorie de produit sur les 3 dernières années ? Quels mois sont les plus rentables ? Quels clients fidèles méritent une attention particulière ?"*

L'équipe technique lance la requête directement sur la base opérationnelle. Résultat :
- ⏳ La requête tourne pendant **6 minutes**
- 🔴 Le site Mexora **ralentit**, les clients ne peuvent plus passer commande
- 💥 Le serveur **sature**
- 📉 Les données retournées sont **incohérentes** car issues de plusieurs sources non harmonisées

Ce moment illustre le problème fondateur de la Business Intelligence : les systèmes conçus pour *enregistrer* les données ne sont architecturalement pas conçus pour les *analyser*.

La solution : séparer les deux mondes

La BI naît de cette séparation nécessaire entre **faire tourner le business** (système opérationnel) et **analyser le business** (système décisionnel). On crée un environnement dédié à l'analyse, alimenté par les sources opérationnelles mais totalement indépendant d'elles.

> 💡 **Analogie** : Un cuisinier (opérationnel) prépare les plats à la chaîne. Un critique gastronomique (décisionnel) analyse et juge les plats. Le mettre derrière les fourneaux bloquerait tout le service.

1.2 Historique — De l'Infocentre au Data Engineering

Comprendre d'où vient la BI permet de comprendre *pourquoi* elle est architecturée comme elle l'est aujourd'hui. Chaque évolution répond à une limite concrète de la génération précédente.

Années 1960-70 — MIS : les premiers rapports informatisés

Les **MIS** (Management Information Systems) produisent des rapports périodiques standardisés pour les managers à partir des bases de production. Ils sont rigides, non interactifs, et ne permettent que des requêtes prédéfinies. Le décideur reçoit ce qu'on a décidé de lui donner — il ne peut pas explorer.

Limite : Impossible de répondre à une question non anticipée par le concepteur du système.

Années 1970-80 — DSS : l'aide à la décision interactive

Concept fondateur de **Gorry & Scott Morton (1971)** : créer un système *interactif* qui aide les décideurs à résoudre des problèmes *semi-structurés* en combinant données, modèles analytiques et jugement humain.

> Un problème **semi-structuré** est un problème pour lequel on dispose de certaines données mais où le jugement humain reste indispensable — contrairement aux problèmes entièrement algorithmiques.

Exemple Mexora DSS : Décider si Mexora doit ouvrir un entrepôt à Agadir. On peut modéliser les coûts et les volumes, mais la décision finale intègre des facteurs humains, politiques et stratégiques non quantifiables.

Limite : Les DSS sont construits sur les mêmes bases de données que les systèmes opérationnels — les requêtes analytiques lourdes bloquent les transactions. C'est exactement le problème de Mexora au début de ce chapitre.

Années 1980 — Infocentre et EIS

L'Infocentre est une évolution intelligente : on crée une *copie périodique* des bases de production, dédiée aux analystes. Les analystes n'accèdent plus aux données de production — ils interrogent une copie. C'est l'ancêtre direct du Data Warehouse.

Les **EIS** (Executive Information Systems) apparaissent avec les premiers PC dans les bureaux des dirigeants. Tableaux permanents d'indicateurs clés (CA, nombre d'employés, parts de marché) — mais les mécanismes d'agrégation et de calcul sont complexes à maintenir.

1989 — Howard Dresner invente le terme "Business Intelligence"

L'analyste de Gartner **Howard Dresner** formule pour la première fois le terme "Business Intelligence" pour décrire :

> *"Un ensemble de concepts et méthodes pour améliorer la prise de décision en utilisant des systèmes à base de faits."*

La BI n'est pas un concept nouveau — c'est une reformulation commerciale et élargie du DSS, enrichie de l'infrastructure technique qui émerge à cette époque.

Années 1990-2000 — L'âge d'or du Data Warehouse

Deux figures dominent cette période et structurent encore l'enseignement de la BI aujourd'hui :

- **Bill Inmon** (1992) — publie *"Building the Data Warehouse"*. Il définit le DWH comme une collection de données thématiques, intégrées, non-volatiles et historisées. Approche **Top-Down** : d'abord le DWH central, puis les Data Marts.
- **Ralph Kimball** (1996) — publie *"The Data Warehouse Toolkit"*. Il popularise la **modélisation dimensionnelle** (faits, dimensions, schéma en étoile). Approche **Bottom-Up** : d'abord les Data Marts métier, qui forment progressivement le DWH.

Ce débat Inmon vs Kimball est encore vivant en 2025 dans les équipes data.

Années 2000-2010 — La BI self-service

Les outils se démocratisent. Business Objects, Cognos, puis Tableau et Power BI permettent aux analystes non-techniques de créer leurs propres rapports sans dépendre de l'IT. C'est la "BI self-service".

2010s — Big Data et Data Lake

Le volume, la variété et la vélocité des données explosent. Hadoop, Spark, et les Data Lakes émergent pour traiter ce que les DWH classiques ne peuvent pas absorber. La discipline du **Data Engineering** se professionnalise.

2020s — IA, LLMs et Data Lakehouse

L'IA générative transforme chaque couche du pipeline. Le Data Lakehouse converge les paradigmes. Le rôle du Data Engineer s'élargit vers le MLOps et les pipelines d'IA.

```
MIS (1960s) → DSS (1971) → Infocentre (1980s) → EIS (1980s)
     → BI / DWH (1989-2000) → BI self-service (2000s)
     → Big Data / Data Lake (2010s) → Data Lakehouse + IA (2020s)
```

1.3 Définitions fondamentales

Business Intelligence (BI)

> L'**informatique décisionnelle** (Business Intelligence ou BI) désigne les moyens, les outils et les méthodes qui permettent de **collecter**, **consolider**, **modéliser** et **restituer** les données d'une entreprise en vue d'offrir une aide à la décision et de permettre aux responsables stratégiques d'avoir une vue d'ensemble de l'activité traitée.

Decision Support System (DSS)

> Un **DSS** (Système d'Aide à la Décision) est un système informatique interactif qui aide les décideurs à résoudre des problèmes **semi-structurés** en combinant des données, des modèles analytiques et des outils de visualisation. Il offre la possibilité de modélisation et de simulation, et fournit un accès aisé aux informations via des outils spécialisés.

Exemple Mexora : Un DSS permet au Directeur Commercial de visualiser en temps réel les ventes par région, d'identifier les produits à faible rotation à

Tanger vs Casablanca, et de décider des promotions à lancer — sans solliciter l'équipe technique à chaque fois.

De la Donnée à la Décision — La pyramide DIKW

Avant d'aller plus loin, il faut poser une distinction fondamentale que toute l'architecture BI cherche à résoudre : **la différence entre une donnée et une information**.

La Donnée

> Une **donnée** est un fait brut, isolé, sans contexte. Elle n'a pas de sens en elle-même. Elle est le résultat d'une observation ou d'une mesure.

Exemples chez Mexora :
- `5` — un nombre
- `2024-11-15` — une date
- `TNG` — un code
- `899.00` — un montant
- `3` — une quantité

Ces données stockées dans la base MySQL de Mexora ne disent rien sans contexte. Elles sont les atomes de l'information.

L'Information

> Une **information** est une donnée **contextualisée et interprétée** selon plusieurs dimensions. Elle acquiert un sens et répond à une question.

> 💡 Par **exemple :** `5°C` est une donnée. `Il fait froid ce matin à Tanger` est une information — on a contextualisé la donnée brute selon les dimensions *lieu* et *temps*, et on l'a interprétée par rapport à une référence humaine (le confort thermique).

Même logique chez Mexora :

Données brutes	+ Dimensions	= Information
`899.00`	+ produit "iPhone 16" + client "Karim" + date "15/11" + ville "Tanger"	→ "Karim a acheté un iPhone 16 à 899 MAD à Tanger le 15 novembre"

Données brutes	+ Dimensions	= Information
`3`	+ “commandes annulées” + “région Rabat” + “T3 2024”	→ “3 commandes ont été annulées à Rabat au 3ème trimestre 2024”
`0.72`	+ “taux de marge” + “catégorie Alimentation” + “mois de Ramadan”	→ “La marge sur l’alimentation chute à 72% pendant le Ramadan”

> **Conséquence architecturale directe :** C’est exactement pourquoi le schéma en étoile existe. La **table de faits** stocke les données brutes mesurables (899.00, 3, 0.72). Les **tables de dimensions** fournissent le contexte (qui, quoi, où, quand). Ensemble, ils transforment des données en informations exploitables. Le modèle dimensionnel n’est pas une convention arbitraire — c’est la traduction technique de cette chaîne Donnée → Information.

La Connaissance

> La **connaissance** est l’accumulation et la structuration d’informations dans le temps, qui crée une compréhension durable des phénomènes. Elle répond non plus à “que s’est-il passé ?” mais à “pourquoi ?” et “que va-t-il se passer ?”.

Exemple Mexora :
L’information “les ventes d’alimentation augmentent pendant le Ramadan” observée sur 3 années consécutives devient une **connaissance** : “le pic Ramadan est structurel et prévisible, il survient chaque année avec une hausse moyenne de +34% sur l’alimentation”.

Cette connaissance permet d’anticiper — constituer des stocks avant le Ramadan, recruter des livreurs temporaires, lancer des campagnes promotionnelles ciblées.

La Décision (Wisdom - La sagesse)

> La **décision** est la mobilisation de la connaissance pour **agir**. Elle intègre la connaissance analytique et le jugement humain pour trancher entre plusieurs options.

Exemple Mexora :
La connaissance du pic Ramadan + la connaissance de la capacité logistique +

le jugement du directeur → **Décision** : augmenter le stock d'alimentation de 40% dès 3 semaines avant le début du Ramadan, et recruter 15 livreurs temporaires.

La pyramide DIKW — Russell Ackoff (1989)

Cette hiérarchie est formalisée par le chercheur **Russell Ackoff** sous le nom de pyramide **DIKW** (Data → Information → Knowledge → Wisdom). Elle date de la même époque que les premiers DWH et les DSS — ce n'est pas un hasard.

```
      ┌──────────────┐
      │   DÉCISION   │  Agir — jugement humain
      │   Wisdom     │  "Augmenter les stocks de 40%"
      ├──────────────┤
      │ CONNAISSANCE │  Comprendre — patterns dans le temps
      │  Knowledge   │  "Le pic Ramadan est structurel"
    ─┤──────────────├─
      │ INFORMATION  │  Interpréter — données contextualisées
      │ Information  │  "Les ventes ont augmenté de 34%"
      ├──────────────┤
      │   DONNÉES    │  Observer — faits bruts
      │     Data     │  "5°C", "899.00 MAD", "3 retours"
      └──────────────┘
Volume décroissant ↑  Valeur décisionnelle croissante ↑
```

La BI s'occupe des deux couches du milieu : transformer des données brutes en informations (ETL + DWH), et structurer ces informations pour construire de la connaissance (modélisation dimensionnelle, OLAP, dashboards). La décision finale reste humaine.

> ⚠️ **Point critique :** Un système BI produit de l'**information**, pas de la décision. Confondre les deux est la source de nombreuses déceptions post-implémentation — "on a installé un outil BI mais les décisions ne sont pas meilleures". L'outil ne décide pas à la place des managers, il les informe mieux.

Relation entre BI et DSS — Point crucial

Ces deux termes sont souvent confondus ou utilisés de façon interchangeable. Voici la relation exacte :

```
DSS (concept académique, 1971)
    |
    |  ← la BI est l'implémentation industrielle du DSS
```

```
    ▼
BI (terme industriel, 1989)
    │
    │  ← la BI s'appuie sur cette infrastructure
    ▼
Data Warehouse + ETL + OLAP
```

Critère	DSS	BI
Nature	Concept académique	Ensemble d'outils et méthodes industriels
Origine	Gorry & Scott Morton, 1971	Howard Dresner / Gartner, 1989
Périmètre	Aide à la décision (large)	Collecte, consolidation, restitution de données
Infrastructure	Peut être un simple tableur	Requiert DWH, ETL, outils de visualisation
Relation	Le concept général	L'implémentation technique concrète

> **En résumé :** Tout système BI est un DSS. Tout DSS n'est pas forcément une BI. La BI est la réalisation technique et industrielle de l'objectif intellectuel du DSS. Le DSS est le *pourquoi*, la BI est le *comment*.

Les outils du système décisionnel

Un système décisionnel complet chez Mexora s'appuie sur plusieurs niveaux d'outils :

EIS / SIAD (héritage des années 80-90)
Un **EIS** organise et met en forme des indicateurs dans des tableaux de bord. Simple à utiliser mais limité aux requêtes prédéfinies. Un **SIAD** (Système Informatisé d'Aide à la Décision) permet la modélisation multidimensionnelle plus complexe mais nécessite un apprentissage plus lourd.

Outils OLAP et Reporting
Permettent l'analyse multidimensionnelle interactive (Roll-up, Drill-down, Slice, Dice). Voir Chapitre 4.

Data Mining
Met en évidence des corrélations dans un grand volume de données. S'appuie sur des techniques d'IA (réseaux de neurones, arbres de décision) pour dégager des tendances cachées. Exemple Mexora : détecter que les clients qui achètent de l'électronique en septembre achètent aussi de la mode en octobre.

Moteur de règles de gestion
Capitalise l'expertise et le métier de l'entreprise sous forme de règles automatisables. Le décisionnel aide dans la conception des processus décisionnels mais **ne se substitue pas aux décideurs**.

1.4 OLTP vs OLAP — La rupture fondamentale

Ce sont deux paradigmes **opposés** qui coexistent dans toute entreprise. Les confondre génère des architectures défaillantes et des performances catastrophiques.

OLTP — Online Transaction Processing

> **OLTP** désigne les systèmes conçus pour gérer des transactions courantes en temps réel. Chaque opération touche peu de lignes, mais doit s'exécuter en millisecondes.

Les SI traditionnels sont des systèmes opérationnels créés pour répondre aux besoins du traitement de transactions en ligne. Ils sont construits pour les opérations du jour-le-jour et ne se préoccupent pas de la compilation ou de l'historisation des données dans le temps.

Exemple Mexora OLTP :

```sql
-- Transaction typique : enregistrer une commande en temps
réel
INSERT INTO commandes (id_client, date_commande, statut)
VALUES (1042, NOW(), 'en_attente');

INSERT INTO lignes_commande (id_commande, id_produit, quant
ite, prix_unitaire)
VALUES (5891, 234, 2, 899.00);
-- Durée attendue : < 50 millisecondes
```

OLAP — Online Analytical Processing

> **OLAP** désigne les systèmes conçus pour l'analyse de grandes volumes de données historiques. Les requêtes touchent des millions de lignes et sont tolérantes sur le temps de réponse.

Les systèmes OLAP visent à combler les lacunes des systèmes transactionnels. Les utilisateurs n'ont aucun besoin de modifier les données — ils veulent analyser, croiser, agréger, comparer dans le temps.

Exemple Mexora OLAP :

```sql
-- Requête analytique : CA trimestriel par région et catégo
rie
SELECT
    r.region_admin,
    p.categorie,
    t.trimestre,
    t.annee,
    SUM(f.montant_ttc)          AS ca_total,
    COUNT(DISTINCT f.id_client) AS nb_clients,
    AVG(f.montant_ttc)          AS panier_moyen
FROM fait_ventes f
JOIN dim_region  r ON f.id_region  = r.id_region
JOIN dim_produit p ON f.id_produit = p.id_produit
JOIN dim_temps   t ON f.id_date    = t.id_date
GROUP BY r.region_admin, p.categorie, t.trimestre, t.annee
ORDER BY ca_total DESC;
-- Durée acceptable : quelques secondes
```

Tableau comparatif complet

Critère	OLTP (Mexora opérationnel)	OLAP (Mexora analytique)
Objectif	Enregistrer commandes, paiements	Analyser ventes, tendances
Opérations	INSERT / UPDATE / DELETE	SELECT avec agrégats
Données	Actuelles, temps réel	Historiques, consolidées
Utilisateurs	Vendeurs, logisticiens	Directeurs, analystes
Modèle	Normalisé 3NF	Dénormalisé (schéma étoile)
Volume/requête	Quelques lignes	Des millions de lignes
Temps de réponse	Millisecondes (critique)	Secondes (acceptable)
Mises à jour	Interactives et fréquentes	Aucune après chargement

Critère	OLTP (Mexora opérationnel)	OLAP (Mexora analytique)
Redondance	Aucune (normalisé)	Acceptée (performances)
Conservation	État actuel uniquement	Historique sur plusieurs années

1.5 Architecture globale d'un système BI

L'informatique décisionnelle s'insère dans l'architecture plus large du Système d'Information de l'entreprise. Elle utilise en règle générale un **entrepôt de données** pour stocker des données transverses provenant de plusieurs sources hétérogènes et fait appel à des **traitements par lots** pour la collecte de ces informations.

```
┌──────────────────────────────────────────┐
│  SOURCES MEXORA (Système opérationnel)   │
│  MySQL (commandes) | Excel (stocks)      │
│  API paiements CMI | Fichiers livreurs   │
│  CRM | Réseaux sociaux                   │
└──────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────┐
│  ETL / ELT                               │
│  Extraction → Transformation → Chargement │
│  (Nettoyage, harmonisation, enrichissement) │
└──────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────┐
│  DATA WAREHOUSE                          │
│  PostgreSQL — Modèle dimensionnel        │
│  Données historiques consolidées         │
│                                          │
│  ┌────────────────┐  ┌────────────────┐  │
│  │  Data Mart     │  │  Data Mart     │  │
│  │  Commercial    │  │  Logistique    │  │
│  └────────────────┘  └────────────────┘  │
└──────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────┐
│  RESTITUTION                             │
│  Power BI | Metabase | Rapports SQL      │
│  Alertes automatiques | Exports          │
└──────────────────────────────────────────┘
```

Rôle des Data Marts

> Un **Data Mart** est un sous-ensemble de l'entrepôt de données, dédié à un métier particulier de l'entreprise (marketing, logistique, finance...). Il est généralement alimenté depuis le Data Warehouse central.

Chez Mexora :
- **Data Mart Commercial** : ventes, clients, produits, remises
- **Data Mart Logistique** : livraisons, retours, stocks, délais
- **Data Mart Finance** : marges, coûts, facturation fournisseurs

---

1.6 Les 4 étapes du processus décisionnel

1. La Collecte

Aller à la recherche des données dans les sources opérationnelles de Mexora. Les données peuvent être collectées sous deux optiques : **au fil du temps** (historique des transactions) ou **à un instant T** (snapshot de l'état de l'entreprise). L'outil technique utilisé pour cette collecte est l'**ETL**.

2. L'Intégration

Structurer les données utiles à la décision en unifiant l'espace de données. Les données sont épurées et transformées par un filtrage et une validation, une synchronisation (aligner des événements déphasés) et une certification (rapprocher avec les données légales et comptables).

Exemple Mexora : La ville "Tanger" peut apparaître comme "tanger", "TNG", "TANGER" ou "Tanja" selon les systèmes sources. L'intégration harmonise tout en "Tanger".

3. La Diffusion

Mettre les données à disposition selon les profils utilisateurs. Chaque Data Mart de Mexora est segmenté selon les besoins de ses utilisateurs cibles, avec des agrégats précalculés adaptés au niveau de détail requis.

4. La Restitution

Présenter les informations de la façon la plus lisible possible pour le décideur. On modélise des représentations à base de requêtes — c'est le **reporting**. Cela inclut la sélection de critères de filtre, le tri et regroupement des données, les calculs et les diagrammes.

1.7 Les acteurs et leurs rôles chez Mexora

🔧 **Data Engineer**
Construit les pipelines ETL, maintient l'entrepôt de données, garantit la fraîcheur et la fiabilité des données. C'est le "plombier de la donnée" — invisible quand tout fonctionne, crucial quand ça tombe.

📊 **Data Analyst**
Exploite le DWH pour produire les rapports du CODIR mensuel, répond aux questions métier, crée les dashboards. Il parle à la fois aux données et aux décideurs.

🤖 **Data Scientist**
Prédit les ruptures de stock, segmente les clients, recommande des produits. Il utilise les données du DWH comme matière première pour ses modèles ML.

👔 **Business User (Directeur Mexora)**
Consomme les dashboards, formule les besoins analytiques, prend les décisions. Il ne touche jamais aux données brutes.

> ⚠️ **Erreur fréquente :** Le système décisionnel aide à la prise de décision mais **ne se substitue pas aux décideurs**. Il fournit les informations, les humains décident.

✅ À retenir — Chapitre 1

> **1.** Une **donnée** est un fait brut sans contexte. Une **information** est une donnée contextualisée selon plusieurs dimensions. La BI transforme les données en informations — jamais directement en décisions.
>
> **2.** La pyramide **DIKW** (Donnée → Information → Connaissance → Décision) justifie toute l'architecture BI : la table de faits stocke les données, les dimensions fournissent le contexte, ensemble ils produisent l'information.
>
> **3.** La BI naît d'une séparation nécessaire : les systèmes OLTP (opérationnels) et OLAP (analytiques) ont des architectures incompatibles — mélanger les deux dégrade les deux systèmes.
>
> **4.** Le chemin universel de tout système BI est : **Source** → **ETL** → **Data Warehouse** → **Data Mart** → **Restitution**. Chaque étape transforme les données vers un niveau d'information supérieur.

✅ Key takeaways — Chapter 1

> **1.** A **data point** is a raw fact without context. **Information** is data placed into context across multiple dimensions. Business Intelligence turns data into information, never directly into decisions.
>
> **2.** The **DIKW** pyramid (Data → Information → Knowledge → Wisdom) explains BI architecture: the **fact table** stores measurable data, **dimension tables** provide context, and together they produce information.
>
> **3.** BI comes from a necessary separation: **OLTP** (operational) and **OLAP** (analytical) systems have incompatible architectures. Mixing them degrades both systems.
>
> **4.** The universal path of any BI system is: **Source** → **ETL** → **Data Warehouse** → **Data Mart** → **Reporting/Delivery**. Each step transforms data into a higher level of usable information.

Chapitre 2 — Modélisation Dimensionnelle | Dimensional Modeling

2.1 Pourquoi abandonner le modèle relationnel classique ?

La base opérationnelle de Mexora est en 3NF : 52 tables, une normalisation parfaite conçue pour éliminer la redondance. C'est excellent pour les transactions.

Mais pour analyser les ventes, il faut joindre 14 tables. La requête est illisible, lente, et incompréhensible pour un analyste non-technique. Comme le dit Ralph Kimball (1997) :

> *"Les modèles entité-relation ne peuvent pas servir de base pour construire des entrepôts de données de l'entreprise."*

La **modélisation multidimensionnelle** (Codd, 1993) est une alternative au modèle relationnel. Elle présente les données non plus sous forme de tables normalisées mais de **cubes centrés sur une activité**, conçus pour les analyses

décisionnelles. Elle sacrifie volontairement la normalisation pour gagner en lisibilité et en performances analytiques.

2.2 Les concepts fondamentaux

Le Fait

> Un **fait** est un centre d'intérêt décisionnel. Il enregistre les **événements mesurables** du business. Chaque ligne représente un fait qui s'est produit (une vente, un retour, une livraison). Il regroupe un ensemble d'attributs numériques représentant les **mesures** d'activité.

Règle absolue : Une table de faits ne contient jamais de texte descriptif. Si vous trouvez un nom de produit dans votre table de faits, votre modèle est incorrect.

Les Mesures

> Une **mesure** est un indicateur d'analyse de type numérique et cumulable. Elle est accompagnée de fonctions d'agrégation (SUM, AVG, MIN, MAX) qui permettent de l'agréger en fonction des axes d'analyse.

Propriétés des mesures — point crucial :

Type	Définition	Exemple Mexora
Additive	Sommable sur toutes les dimensions	Montant TTC, Quantité vendue, CA
Semi-additive	Sommable sur certaines dimensions seulement	Niveau de stock (pas sommer dans le temps)
Non-additive	Ne se somme pas, doit être recalculée	Taux de marge, Prix unitaire

> ⚠️ **Piège classique :** Le taux de remise moyen de Mexora ne s'obtient pas en faisant la moyenne des taux de remise — c'est une mesure **non-additive**. Il faut recalculer : `SUM(montant_remise) / SUM(montant_avant_remise)`.

Les Dimensions

> Une **dimension** est un axe d'analyse selon lequel sont visualisées les mesures d'activité. Elle contient les attributs textuels et descriptifs qui permettent de filtrer et regrouper les analyses.

Les 5 dimensions classiques : **Temps, Produit, Client, Lieu, Canal**

Les Paramètres et Hiérarchies

> Un **paramètre** est un attribut appartenant à une dimension — il représente un niveau de détail selon lequel sont visualisées les mesures.

> Une **hiérarchie** est une perspective d'analyse définie dans une dimension. Elle organise les paramètres de la granularité la plus fine vers la plus générale.

Exemple — Dimension Temps chez Mexora :

```
All (Total)
  └── Année (2024)
        └── Trimestre (T2)
              └── Mois (Avril)
                    └── Semaine
                          └── Jour  ← granularité la plus fine
```

Les attributs comme `lib_mois` ("Avril"), `est_ferie_maroc` sont des **attributs faibles** — ils décrivent un paramètre mais ne font pas partie des calculs de regroupement.

La Granularité

> La **granularité** est le niveau de détail le plus fin représenté dans la table de faits. Elle répond à la question : *"Qu'est-ce qu'une ligne de cette table représente exactement ?"*

C'est la décision architecturale la plus importante de toute la modélisation.

Exemple Mexora :
- Granularité "ligne de commande" → on peut analyser produit par produit ✅
- Granularité "commande" → impossible d'analyser par catégorie de produit ❌

2.3 Les schémas multidimensionnels

Schéma en Étoile (Star Schema)

> Le **schéma en étoile** est constitué d'**1 table de faits centrale** reliée à **n tables de dimensions dénormalisées**. C'est la forme la plus simple et la plus

> utilisée en BI.

Avantages :
- Navigation facile et intuitive
- Performances : nombre de jointures limité
- Gestion efficace des agrégats
- Fiabilité des résultats

Inconvénients :
- Redondance dans les dimensions
- Alimentation plus complexe

Modèle Mexora — Schéma en étoile complet :

```
                      DIM_TEMPS
                      ----------
                      id_date (PK)
                      jour | mois | trimestre | annee
                      libelle_mois | est_weekend
                      est_ferie_maroc | periode_ramadan
                            |
DIM_CLIENT           FAIT_VENTES            DIM_PRODUIT
----------           -----------            -----------
id_client (PK)  ←── id_date      (FK)  ──► id_produit (PK)
nom_complet          id_produit   (FK)      nom_produit
age | sexe           id_client    (FK)      categorie
ville_residence      id_region    (FK)      sous_categorie
segment_client       id_livreur   (FK)      marque | fournisseur
date_inscription     ─────────────         prix_standard
canal_acquisition    quantite_vendue        origine_pays
                     montant_ht
                     montant_ttc
                     remise_pct
                     cout_livraison
                     delai_livraison_j
                            |
                   DIM_LIVREUR        DIM_REGION
                   -----------        ----------
                   id_livreur (PK)    id_region (PK)
                   nom_livreur        ville | province
                   type_transport     region_admin
                   zone_couverture    zone_geo
                                      pays
```

Implémentation SQL :

```sql
-- Table de faits Mexora
CREATE TABLE fait_ventes (
```

```
    id_vente                SERIAL PRIMARY KEY,
    id_date                 INTEGER NOT NULL REFERENCES dim_t
emps(id_date),
    id_produit              INTEGER NOT NULL REFERENCES dim_p
roduit(id_produit),
    id_client               INTEGER NOT NULL REFERENCES dim_c
lient(id_client),
    id_region               INTEGER NOT NULL REFERENCES dim_r
egion(id_region),
    id_livreur              INTEGER REFERENCES dim_livreur(id
_livreur),
    quantite_vendue         INTEGER NOT NULL,
    montant_ht              DECIMAL(12,2) NOT NULL,
    montant_ttc             DECIMAL(12,2) NOT NULL,
    remise_pct              DECIMAL(5,2) DEFAULT 0,
    cout_livraison          DECIMAL(8,2),
    delai_livraison_jours SMALLINT
);
```

Schéma en Flocon de Neige (Snowflake Schema)

> Le **schéma en flocon** est une extension du schéma en étoile dans lequel les dimensions sont **normalisées** en sous-tables. Lorsque les dimensions sont trop volumineuses, on les décompose.

Avantages : Réduction du volume, analyse par palier (drill-down) facilitée sur les dimensions hiérarchisées.

Inconvénients : Navigation difficile, nombreuses jointures.

Exemple — Normalisation de DIM_PRODUIT chez Mexora :

```
Schéma étoile (dénormalisé) :
DIM_PRODUIT (id_produit, nom, categorie, sous_categorie, marque)

Schéma flocon (normalisé) :
DIM_PRODUIT (id_produit, nom, id_sous_categorie, id_marque)
    └── DIM_SOUS_CATEGORIE (id_sous_cat, libelle, id_categorie)
            └── DIM_CATEGORIE (id_categorie, libelle)
    └── DIM_MARQUE (id_marque, nom_marque, pays_origine)
```

> **Recommandation pratique :** Dans 90% des projets BI réels, **l'étoile bat le flocon**. La performance et la lisibilité priment sur la pureté théorique. Choisissez le flocon uniquement si la redondance de données représente un coût de stockage significatif — ce qui est rare avec les capacités actuelles.

Schéma en Constellation

> Le **schéma en constellation** est une généralisation du schéma en étoile : il regroupe **plusieurs faits** reliés à des dimensions éventuellement **partagées entre ces faits**.

Avantages :
- Facilite les corrélations entre sujets d'analyse
- Simplifie la modélisation en partageant les dimensions communes

Exemple Mexora — Constellation Ventes + Livraisons :

```
FAIT_VENTES               FAIT_LIVRAISONS
-----------               ---------------
id_date    (FK) ──────────── id_date    (FK)
id_produit (FK) ──────────── id_produit (FK)
id_client  (FK)               id_livreur (FK)
id_region  (FK) ──────────── id_region  (FK)
montant_ttc                   cout_livraison
quantite                      delai_jours
                              statut_livraison

   ↑↑ DIM_DATE, DIM_PRODUIT, DIM_REGION sont partagées ↑↑
```

2.4 Méthodes de conception d'un schéma multidimensionnel

Méthode Descendante (Top-Down)

> La démarche **descendante** part des **besoins des décideurs** pour concevoir le schéma multidimensionnel. Elle se base sur les requêtes-types des utilisateurs.

Les 3 étapes :

1. Collecte des données
Interviewer les décideurs de Mexora pour collecter leurs requêtes-types. Une

requête-type se structure en 3 clauses :

```
Analyser  [quoi ?]      : le montant des ventes
En fonction de [axes ?] : la région, la catégorie de produit et le mois
Pour [filtres ?]        : les produits électroniques uniquement
```

2. Spécification des besoins — Matrice des besoins
Construire une matrice où les lignes = mesures à analyser, les colonnes = paramètres d'analyse. Chaque case cochée indique qu'une mesure est analysée en fonction d'un paramètre.

```
Mexora — Matrice des besoins :
              | Mois | Région | Catégorie | Livreur | Client |
Montant TTC   |  ✓   |   ✓    |     ✓     |         |   ✓    |
Quantité      |  ✓   |   ✓    |     ✓     |         |        |
Délai livr.   |  ✓   |   ✓    |           |    ✓    |        |
Coût livr.    |  ✓   |        |           |    ✓    |        |
```

3. Formalisation : La matrice génère directement les faits (lignes = mesures regroupées) et les dimensions (colonnes = paramètres).

Méthode Ascendante (Bottom-Up)

> La démarche **ascendante** part du **schéma conceptuel de la source** (base opérationnelle) pour construire le schéma dimensionnel. Elle détecte les classes représentatives dans le schéma source.

Les 4 étapes :
1. **Détermination des faits** : identifier les classes qui enregistrent des événements mesurables dans la base source
2. **Détermination des dimensions** : détecter les classes déterminantes de chaque classe représentative
3. **Hiérarchisation** : organiser les paramètres en structures hiérarchiques
4. **Granularité** : définir le niveau de détail le plus fin de chaque dimension

Méthode Mixte (recommandée)

> La méthode **mixte** combine les deux approches précédentes : on applique successivement la démarche ascendante (depuis la source) et la démarche descendante (depuis les besoins), puis on confronte les deux schémas.

Avantages :
- Définition correcte de la granularité
- Intégration de toutes les contraintes sémantiques
- Ajout des mesures calculées demandées par les décideurs

2.5 Slowly Changing Dimensions (SCD)

Le problème

Un client Mexora déménage de Tanger à Casablanca. Un produit change de catégorie. Comment gérer ces changements sans perdre l'historique analytique ?

C'est le problème des **Slowly Changing Dimensions** — les dimensions qui évoluent lentement dans le temps.

SCD Type 1 — Écrasement

> On **remplace** l'ancienne valeur par la nouvelle. Pas d'historique conservé.

```
-- Avant : Fatima habite à Tanger
-- Après SCD Type 1 :
UPDATE dim_client SET ville = 'Casablanca' WHERE id_client
= 2087;
-- ⚠ L'ancienne valeur est perdue définitivement
```

Quand l'utiliser : Corrections d'erreurs de saisie (faute d'orthographe, mauvais format). Jamais pour des changements réels ayant une valeur historique.

SCD Type 2 — Historisation par nouvelles lignes ✅ (Le plus utilisé)

> On **crée une nouvelle ligne** pour la nouvelle valeur, avec des dates de validité. L'ancienne ligne est conservée intacte.

```
-- DIM_CLIENT avec SCD Type 2
id   | id_client_src | nom     | ville        | date_debut | d
ate_fin    | est_actif
----|---------------|--------|-------------|------------|--
----------|----------
```

```
1    | C2087          | Fatima | Tanger       | 2022-01-01 | 2
024-08-31 | FALSE
2    | C2087          | Fatima | Casablanca   | 2024-09-01 | 9
999-12-31 | TRUE
```

Avantage crucial : Les ventes de 2022 et 2023 restent rattachées à Tanger dans les analyses historiques. Les ventes de 2024 sont rattachées à Casablanca. L'analyse géographique est correcte dans le temps.

Quand l'utiliser : Tout changement ayant une valeur analytique — ville, segment client, catégorie produit, zone livreur.

SCD Type 3 — Colonne "ancienne valeur"

> On ajoute une **colonne supplémentaire** pour stocker uniquement l'ancienne valeur.

```
id_client | nom     | ville_actuelle | ville_precedente | da
te_changement
----------|--------|----------------|------------------|---
-------------
2087      | Fatima | Casablanca     | Tanger           | 20
24-09-01
```

Limite : Ne conserve qu'**un seul** changement. Si Fatima déménage encore, on perd Tanger.

Quand l'utiliser : Rarement. Uniquement si on ne veut tracker qu'un seul niveau de changement.

2.6 Exercice de modélisation

Cas : L'Université Abdelmalek Essaâdi souhaite analyser les résultats académiques de ses étudiants sur plusieurs années.

Questions à résoudre :
1. Quelle est la granularité appropriée ? (note par matière ? par module ? par semestre ?)
2. Quels sont les faits mesurables et leurs types d'additivité ?
3. Quelles dimensions sont nécessaires ?

4. Quels attributs SCD faut-il gérer ? (un étudiant peut changer de filière)
5. Proposer le schéma en étoile complet

Correction discutée en séance.

✅ À retenir — Chapitre 2

> **1.** La **granularité** est la décision la plus critique de toute modélisation dimensionnelle — elle détermine définitivement ce qu'on peut et ne peut pas analyser.
>
> **2.** Les **mesures** ont trois types d'additivité : additive (SUM sur tout), semi-additive (SUM sur certains axes), non-additive (impossible à sommer). Confondre ces types produit des résultats analytiques faux.
>
> **3.** Le **schéma en étoile** est à privilégier dans 90% des cas — moins de jointures, meilleures performances, meilleure compatibilité avec les outils BI.
>
> **4.** Les **SCD Type 2** (nouvelles lignes avec dates de validité) sont la solution standard pour conserver l'historique quand les données des dimensions changent.
>
> **5.** La modélisation dimensionnelle n'est pas une simplification — c'est un **paradigme différent**, optimisé pour un usage analytique. La méthode mixte (Top-down + Bottom-up) donne les meilleurs résultats en pratique.

✅ Key takeaways — Chapter 2

> **1. Grain (granularity)** is the most critical decision in dimensional modeling. It determines what you can and cannot analyze.
>
> **2.** Measures have three additivity types: **additive** (can be summed across all dimensions), **semi-additive** (only across some dimensions), and **non-additive** (must be recomputed). Confusing these types produces wrong results.
>
> **3.** The **star schema** should be preferred in most cases: fewer joins, better performance, and better compatibility with BI tools.
>
> **4. SCD Type 2** (new rows with validity dates) is the standard solution to preserve history when dimension attributes change.

> **5.** Dimensional modeling is not a simplification. It is a **different paradigm** optimized for analytics. In practice, a mixed approach (Top-down + Bottom-up) often works best.

Chapitre 3 — Processus ETL / ELT | ETL / ELT Processes

3.1 Définition et enjeux

> **ETL** signifie **Extract, Transform, Load** (en français : Extraction, Transformation, Chargement — ETC). C'est le processus qui permet de synchroniser massivement des données d'une banque de données vers une autre, en les transformant pour les rendre cohérentes et exploitables dans le Data Warehouse.

L'ETL est **la partie la plus chronophage d'un projet BI** — on estime qu'il représente 60 à 80% du temps total. Pourquoi ? Parce que la réalité des données d'entreprise est chaotique.

L'ETL repose sur 3 composants techniques :
- **Connecteurs** : permettent l'extraction ou l'importation des données de types divers (Oracle, MySQL, SAP, Excel...)
- **Transformateurs** : manipulent les données (agrégations, filtres, conversions, mises en correspondance)
- **Chargement par batch** : exécution planifiée, par exemple tous les soirs à minuit

Réalité Mexora avant ETL :
- "Tanger" apparaît comme "tanger", "TNG", "TANGER", "Tnja" selon la source
- Les dates sont en `DD/MM/YYYY` dans MySQL et `YYYY-MM-DD` dans le CRM
- Certaines commandes n'ont pas de région associée
- Des produits supprimés ont encore des ventes historiques
- Des clients sont enregistrés en doublon avec des emails différents

---

3.2 Extract — L'Extraction

Définition

> L'**extraction** consiste à lire et récupérer les données des systèmes sources sans les modifier ni perturber les activités de production. Les données extraites doivent être marquées par horodatage afin de pouvoir être pistées.

Types de sources chez Mexora

```
Sources structurées :
├── Base MySQL (commandes, clients, produits)
├── Fichiers CSV (exports comptabilité)
└── Excel (stocks, tarifs fournisseurs)

Sources semi-structurées :
├── API REST (paiements CMI/Stripe, météo)
├── Fichiers JSON (logs application mobile)
└── XML (EDI fournisseurs internationaux)

Sources non structurées :
├── Emails (retours clients, réclamations)
└── Avis et commentaires (scraping)
```

Deux stratégies d'extraction

Extraction complète (Full Load)
On extrait *toutes* les données à chaque exécution. Simple mais coûteux sur de gros volumes.

```python
import pandas as pd
import sqlalchemy

engine = sqlalchemy.create_engine("mysql://user:pass@host/mexora_prod")

# Extraction complète — toutes les commandes
df_commandes = pd.read_sql("SELECT * FROM commandes", engine)
print(f"Mexora :{len(df_commandes)} commandes extraites")
```

Extraction incrémentale (Incremental Load) ✅ Recommandée en production
On extrait uniquement les données nouvelles ou modifiées depuis la dernière exécution.

```
from datetime import datetime

# Dernière date de chargement stockée en métadonnées
last_run = datetime(2024, 11, 30, 23, 59, 59)

query = """
    SELECT * FROM commandes
    WHERE date_modification >%(last_run)s
    ORDER BY date_modification
"""
df_nouvelles = pd.read_sql(query, engine, params={"last_ru
n": last_run})
print(f"Mexora :{len(df_nouvelles)} nouvelles commandes dep
uis le{last_run}")
```

3.3 Transform — La Transformation

Définition

> La **transformation** est l'ensemble des opérations qui **nettoient, standardisent, enrichissent et homogénéisent** les données pour les rendre cohérentes dans le DWH. C'est le cœur du travail ETL — et la phase la plus complexe.

L'objectif : avoir des données homogènes pouvant être traitées de façon cohérente.

Les transformations essentielles chez Mexora

Standardisation — harmoniser les formats

```
import pandas as pd

# Problème : la ville est codée différemment selon la sourc
e
# Appli 1 : "tanger"   |  Appli 2 : "TNG"  |  Appli 3 : "TA
NGER"
# Solution : table de correspondance
```

```
mapping_villes = {
    'tanger': 'Tanger', 'TNG': 'Tanger', 'TANGER': 'Tange
r', 'Tnja': 'Tanger',
    'casa': 'Casablanca', 'CASABLANCA': 'Casablanca', 'cb
l': 'Casablanca',
    'rbat': 'Rabat', 'RABAT': 'Rabat',
    'fes': 'Fès', 'FES': 'Fès', 'fez': 'Fès'
}
df['ville'] = df['ville'].str.strip().replace(mapping_ville
s)

# Standardisation des dates (multiple formats)
df['date_commande'] = pd.to_datetime(df['date_commande'], f
ormat='mixed', dayfirst=True)

# Standardisation des genres (3 sources différentes)
# Source 1: m/f  |  Source 2: 1/0  |  Source 3: Homme/Femme
mapping_genre = {'1': 'm', '0': 'f', 'Homme': 'm', 'Femme':
'f', 'homme': 'm', 'femme': 'f'}
df['sexe'] = df['sexe'].replace(mapping_genre)
```

Déduplication — éliminer les doublons

```
# Identifier les clients en doublon sur l'email normalisé
df['email_norm'] = df['email'].str.lower().str.strip()
df_dedup = df.drop_duplicates(subset=['email_norm'], keep
='last')

nb_doublons = len(df) - len(df_dedup)
print(f"Mexora :{nb_doublons} clients en doublon supprimé
s")
```

Gestion des valeurs manquantes — stratégie par colonne

```
# Données critiques → supprimer la ligne
df = df.dropna(subset=['id_commande', 'montant_ttc'])
```

```
# Données géographiques → valeur par défaut
df['region'] = df['region'].fillna('Non renseignée')

# Données numériques → médiane (robuste aux valeurs extrême
s)
df['age_client'] = df['age_client'].fillna(df['age_clien
t'].median())
```

Enrichissement — créer de nouvelles colonnes à valeur analytique

```
# Calcul de la marge brute
df['marge_brute'] = df['montant_ht'] - df['cout_produit']
df['taux_marge_pct'] = (df['marge_brute'] / df['montant_h
t'] * 100).round(2)

# Segmentation clients Mexora (règle métier)
def segmenter_client(ca_cumule: float) -> str:
    if ca_cumule >= 15000:  return 'Gold'
    elif ca_cumule >= 5000: return 'Silver'
    else:                   return 'Bronze'

df['segment'] = df['ca_cumule_12m'].apply(segmenter_client)

# Indicateur période Ramadan (important pour Mexora Maroc)
def est_ramadan(date) -> bool:
    # Logique de calcul basée sur le calendrier hégirien
    return calculer_ramadan(date)

df['periode_ramadan'] = df['date_commande'].apply(est_ramad
an)
```

Filtrage et validation — éliminer les données aberrantes

```
# Éliminer les montants impossibles
df = df[df['montant_ttc'] > 0]
df = df[df['quantite_vendue'] > 0]

# Validation : date de commande ne peut pas être dans le fu
```

```
tur
from datetime import date
df = df[df['date_commande'].dt.date <= date.today()]

# Certification : rapprocher avec la comptabilité
df = df[df['id_commande'].isin(ids_commandes_comptabilite)]
```

3.4 Load — Le Chargement

Définition

> Le **chargement** consiste à insérer les données transformées dans le Data Warehouse. C'est la phase la moins complexe techniquement, mais qui nécessite une stratégie claire pour assurer la cohérence.

Stratégies de chargement

Truncate & Load (chargement complet)
On vide la table et on recharge tout. Simple, garantit la cohérence, mais coûteux.

```
TRUNCATE TABLE fait_ventes;
-- Puis insertion complète
```

Upsert (chargement incrémental) ✅ Recommandé
On insère les nouvelles lignes et on met à jour les existantes si nécessaire.

```
-- PostgreSQL : UPSERT efficace avec ON CONFLICT
INSERT INTO fait_ventes (id_vente, id_date, id_produit, id_
client, montant_ttc, quantite_vendue)
VALUES (%(id)s, %(date)s, %(produit)s, %(client)s, %(montan
t)s, %(qte)s)
ON CONFLICT (id_vente)
DO UPDATE SET
    montant_ttc    = EXCLUDED.montant_ttc,
    quantite_vendue = EXCLUDED.quantite_vendue;
```

Politique de rafraîchissement
L'entrepôt est évolutif. On doit définir la fréquence de mise à jour :
- Batch nocturne (le plus fréquent en BI classique)
- Batch intraday (plusieurs fois par jour pour les KPIs critiques)
- Temps réel streaming (cas particuliers — voir Chapitre 5)

3.5 ETL vs ELT — Le changement de paradigme

Aspect	ETL (traditionnel)	ELT (moderne Cloud)
Ordre	Transformer avant de charger	Charger brut, transformer ensuite
Où ?	Serveur ETL dédié	Dans le DWH lui-même
Avantage	Contrôle total, données propres à l'arrivée	Rapidité, scalabilité Cloud, flexibilité
Inconvénient	Serveur ETL = goulot d'étranglement	Stockage brut coûteux, qualité variable
Outils	Talend, Apache NiFi, Informatica	dbt, BigQuery, Snowflake
Usage	On-premise, volumes maîtrisés	Cloud, big data, Data Lake

> **Mexora aujourd'hui :** Infrastructure on-premise, PostgreSQL. L'ETL Python est le choix adapté. L'ELT deviendrait pertinent si Mexora migrait vers BigQuery ou Snowflake.

3.6 La Data Staging Area (Zone de préparation)

> La **Data Staging Area (DSA)** est une base de données intermédiaire qui stocke les données issues des systèmes de production dans un format proche de leur format source. C'est un **stockage tampon** avant l'intégration dans le DWH.

```
Sources Mexora → [DSA - données brutes] → [Transformations] → [DWH - données propres]
```

Intérêt : Si une transformation échoue, on peut la rejouer sans ré-extraire des sources (qui sont des systèmes de production qu'on ne veut pas surcharger). La DSA préserve aussi la traçabilité (lineage) des données.

✅ À retenir — Chapitre 3

> **1.** L'ETL représente 60 à 80% du temps d'un projet BI. **Ne jamais le sous-estimer** lors d'une estimation de projet — c'est l'erreur la plus fréquente des chefs de projet débutants.
>
> **2.** L'ETL repose sur 3 composants : **connecteurs** (extraire), **transformateurs** (nettoyer/harmoniser), **chargement par batch** (charger selon un planning).
>
> **3.** La qualité du Data Warehouse dépend entièrement des transformations — **"Garbage in, Garbage out"** est une loi immuable de la BI.
>
> **4.** L'extraction **incrémentale** est à privilégier en production : on ne recharge pas ce qui a déjà été correctement chargé — c'est plus rapide et moins intrusif sur les systèmes sources.
>
> **5.** La différence ETL vs ELT n'est pas cosmétique — c'est un choix architectural qui dépend de l'infrastructure disponible (on-premise vs Cloud) et du volume de données.

✅ Key takeaways — Chapter 3

> **1.** ETL typically accounts for **60% to 80%** of a BI project's effort. Never underestimate it when planning.
>
> **2.** ETL relies on three components: **connectors** (extract), **transformers** (clean and standardize), and **batch loading** (scheduled loads).
>
> **3.** Data Warehouse quality depends entirely on transformations: **garbage in, garbage out**.
>
> **4. Incremental extraction** is preferred in production: do not reload data that was already loaded correctly. It is faster and less intrusive on source systems.
>
> **5.** ETL vs ELT is an architectural choice, driven by the available infrastructure (on-premise vs cloud) and data volume.

Chapitre 4 — Data Warehouse avec PostgreSQL | Data Warehousing with PostgreSQL

4.1 Définition formelle d'un Data Warehouse

> D'après **Bill Inmon** (le "père" du Data Warehouse) :
> *"Un entrepôt de données est une collection de données thématiques, intégrées, non-volatiles et historisées, organisées pour la prise de décision."*

Les 4 propriétés fondamentales (Inmon)

Thématiques (orienté sujet)
Les données sont organisées par thèmes métier (ventes, clients, produits) — et non par applications. Cela permet des analyses transversales qui traversent les structures fonctionnelles de Mexora.

Intégrées
Les données proviennent de systèmes sources hétérogènes et sont harmonisées : cohérence sémantique, normalisation des codes, maîtrise des contraintes référentielles.

Non-volatiles
Les données du DWH ne sont pas modifiées après chargement. On ajoute des données, on n'en supprime pas. La traçabilité est garantie.

Historisées
Le DWH conserve l'historique complet des données sur plusieurs années — contrairement aux systèmes OLTP qui ne gardent que l'état actuel.

Différences fondamentales données opérationnelles vs décisionnelles

Données Décisionnelles	Données Opérationnelles
Utilisées de façon aléatoire (requêtes ad hoc)	Utilisées de façon répétitive
Grande quantité de données par traitement	Petite quantité par traitement
Peuvent être redondantes	Uniques (pas de redondance)
Haute disponibilité ponctuelle	Haute disponibilité en continu
Utilisées par l'ensemble des analystes	Accès par une personne à la fois
Pas de mise à jour interactive	Mise à jour interactive possible
Orientées activité/thème, condensées, historiques	Orientées application, détaillées, précises à l'instant T

4.2 Architecture du DWH Mexora sous PostgreSQL

```
-- Organisation en schémas dédiés
CREATE SCHEMA staging_mexora;   -- Zone de préparation (don
nées brutes)
CREATE SCHEMA dwh_mexora;       -- Entrepôt central (modèle
dimensionnel)
CREATE SCHEMA reporting_mexora; -- Vues et agrégats pour le
s outils BI
```

Pourquoi PostgreSQL pour le DWH de Mexora ?

- **Partitionnement natif** : découper les grandes tables par année
- **Vues matérialisées** : précalculer les agrégats lourds
- **Window functions** : calculs analytiques avancés (LAG, RANK, SUM OVER)
- **Index partiels** : indexer uniquement les données récentes et actives
- **Open source** : coût zéro, idéal pour les PME marocaines et le contexte universitaire

---

4.3 Optimisation des performances

Indexation pour la BI

```
-- Index sur les clés étrangères (jointures fréquentes)
CREATE INDEX idx_fait_ventes_date     ON dwh_mexora.fait_ve
ntes(id_date);
CREATE INDEX idx_fait_ventes_produit  ON dwh_mexora.fait_ve
ntes(id_produit);
CREATE INDEX idx_fait_ventes_client   ON dwh_mexora.fait_ve
ntes(id_client);
CREATE INDEX idx_fait_ventes_region   ON dwh_mexora.fait_ve
ntes(id_region);

-- Index composite pour les requêtes les plus fréquentes de
Mexora
CREATE INDEX idx_ventes_date_region
```

```
ON dwh_mexora.fait_ventes(id_date, id_region)
INCLUDE (montant_ttc, quantite_vendue);
```

Partitionnement par date

```
-- Table partitionnée par année — permet d'éviter de scanne
r toutes les années
CREATE TABLE dwh_mexora.fait_ventes (
    id_vente    SERIAL,
    id_date     INTEGER NOT NULL,
    montant_ttc DECIMAL(12,2) NOT NULL
    -- ...autres colonnes
) PARTITION BY RANGE (id_date);

-- Partitions annuelles (format id_date = YYYYMMDD)
CREATE TABLE dwh_mexora.fait_ventes_2022
    PARTITION OF dwh_mexora.fait_ventes
    FOR VALUES FROM (20220101) TO (20230101);

CREATE TABLE dwh_mexora.fait_ventes_2023
    PARTITION OF dwh_mexora.fait_ventes
    FOR VALUES FROM (20230101) TO (20240101);

CREATE TABLE dwh_mexora.fait_ventes_2024
    PARTITION OF dwh_mexora.fait_ventes
    FOR VALUES FROM (20240101) TO (20250101);
```

Vues matérialisées — précalculer les agrégats lourds

> Une **vue matérialisée** stocke physiquement le résultat d'une requête. Contrairement à une vue normale (recalculée à chaque appel), elle est pré-calculée et stockée sur disque. Elle accélère considérablement les dashboards BI.

```
-- Vue matérialisée : CA mensuel par région et catégorie —
Mexora
CREATE MATERIALIZED VIEW reporting_mexora.mv_ca_mensuel AS
```

```sql
SELECT
    t.annee,
    t.mois,
    t.libelle_mois,
    r.region_admin,
    r.zone_geo,
    p.categorie,
    SUM(f.montant_ttc)           AS ca_ttc,
    SUM(f.montant_ht)            AS ca_ht,
    COUNT(DISTINCT f.id_client)  AS nb_clients_actifs,
    SUM(f.quantite_vendue)       AS volume_vendu,
    AVG(f.montant_ttc)           AS panier_moyen,
    SUM(f.montant_ttc - f.montant_ht * 1.2) AS marge_brute_
estimee
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_temps   t ON f.id_date    = t.id_date
JOIN dwh_mexora.dim_region  r ON f.id_region  = r.id_region
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t
GROUP BY t.annee, t.mois, t.libelle_mois, r.region_admin,
r.zone_geo, p.categorie
WITH DATA;

-- Index sur la vue matérialisée pour les filtres courants
CREATE INDEX ON reporting_mexora.mv_ca_mensuel(annee, moi
s);
CREATE INDEX ON reporting_mexora.mv_ca_mensuel(region_admi
n);

-- Rafraîchissement mensuel (à planifier avec pg_cron ou Ai
rflow)
REFRESH MATERIALIZED VIEW CONCURRENTLY reporting_mexora.mv_
ca_mensuel;
```

4.4 Requêtes analytiques OLAP

Roll-up et Drill-down avec GROUPING SETS

> **Roll-up** : passer du détail vers l'agrégat (produit → catégorie → total)
> **Drill-down** : passer de l'agrégat vers le détail (inverse)

```
-- ROLL-UP multi-niveaux — Mexora CA par produit/catégorie/
total
SELECT
    CASE WHEN GROUPING(p.categorie, p.nom_produit) = 0 THEN
p.nom_produit
         WHEN GROUPING(p.categorie) = 0 THEN '[ Total ' ||
p.categorie || ' ]'
         ELSE '[ TOTAL GENERAL ]'
    END AS libelle,
    p.categorie,
    SUM(f.montant_ttc) AS ca_total,
    SUM(f.quantite_vendue) AS volume
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t
WHERE f.id_date BETWEEN 20240101 AND 20241231
GROUP BY GROUPING SETS (
    (p.categorie, p.nom_produit),  -- Niveau détail produit
    (p.categorie),                  -- Niveau catégorie
    ()                              -- Total général
)
ORDER BY p.categorie NULLS LAST, ca_total DESC;
```

Window Functions — Analyses temporelles avancées

```
-- Évolution du CA avec comparaison N-1 et cumul annuel — M
exora
SELECT
    t.annee,
    t.mois,
    t.libelle_mois,
    SUM(f.montant_ttc) AS ca_mois,

    -- Même mois de l'année précédente
    LAG(SUM(f.montant_ttc), 12) OVER (ORDER BY t.annee, t.m
```

```
ois) AS ca_mois_n1,

    -- Évolution en %
    ROUND(
        (SUM(f.montant_ttc) - LAG(SUM(f.montant_ttc), 12) O
VER (ORDER BY t.annee, t.mois))
        / NULLIF(LAG(SUM(f.montant_ttc), 12) OVER (ORDER BY
t.annee, t.mois), 0) * 100,
        2
    ) AS evolution_pct,

    -- Cumul annuel (YTD)
    SUM(SUM(f.montant_ttc)) OVER (
        PARTITION BY t.annee
        ORDER BY t.mois
        ROWS UNBOUNDED PRECEDING
    ) AS ca_ytd

FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_temps t ON f.id_date = t.id_date
GROUP BY t.annee, t.mois, t.libelle_mois
ORDER BY t.annee, t.mois;
```

4.5 Le Cube OLAP — Représentation multidimensionnelle

Définition du cube

> Un **cube OLAP** est une structure de données multidimensionnelle qui organise les mesures d'activité selon plusieurs axes d'analyse (dimensions). Il permet de visualiser et d'interroger les données non plus en lignes de table, mais comme des **points dans un espace à n dimensions**.

La modélisation multidimensionnelle considère les données comme des points dans un espace à plusieurs dimensions. Ces points représentent les centres d'intérêt décisionnels (sujets) analysés en fonction des différents axes d'analyse.

Exemple Mexora — Cube à 3 dimensions :

```
Catégorie de produit
     │  Électronique
     │  Mode
     │  Alimentation
     │
     └──────────────────── Région
        Tanger  Rabat  Casa  Agadir
             \
              \  Temps
               \ T1 T2 T3 T4
                \
                 \
                  [CA TTC] ← mesure au centre du cube
```

Chaque cellule du cube contient une valeur de mesure (ex : CA TTC = 450 000 MAD) pour une combinaison précise de dimensions (Électronique + Tanger + T2 2024).

Avantages du cube OLAP

- **Performances** : les agrégats sont précalculés à tous les niveaux de la hiérarchie
- **Navigation intuitive** : un analyste navigue dans les dimensions sans écrire de SQL
- **Cohérence** : tous les utilisateurs voient les mêmes chiffres depuis la même source

Opérations fondamentales sur un cube

Opération	Définition	Exemple Mexora
Roll-up	Monter dans la hiérarchie — agréger	Jour → Mois → Trimestre → Année
Drill-down	Descendre dans la hiérarchie — détailler	Région → Province → Ville
Slice	Fixer une valeur sur une dimension — plan 2D	“Uniquement les ventes de Tanger”
Dice	Fixer des valeurs sur plusieurs dimensions — sous-cube	“Tanger + Électronique + 2024”
Pivot	Faire pivoter le cube — changer les	Inverser lignes région et

Opération	Définition	Exemple Mexora
	axes d'affichage	colonnes catégorie

4.6 Les trois approches OLAP : ROLAP, MOLAP, HOLAP

Il existe trois façons d'implémenter techniquement un cube OLAP. Le choix entre elles dépend du volume de données, des performances requises et du budget.

ROLAP — Relational OLAP

> **ROLAP** implémente le cube OLAP directement **sur une base de données relationnelle** (PostgreSQL, SQL Server...). Les "cubes" sont en réalité des requêtes SQL complexes avec des agrégations, vues matérialisées et index spécialisés. Il n'y a pas de structure de cube physique séparée.

```
Requête analyste → Moteur ROLAP → SQL → PostgreSQL DWH → Résultat
```

Fonctionnement chez Mexora :

```sql
-- Ce que l'analyste voit : navigation dans un "cube"
-- Ce que ROLAP exécute en coulisse :
SELECT
    r.region_admin,
    p.categorie,
    t.trimestre,
    SUM(f.montant_ttc) AS ca
FROM fait_ventes f
JOIN dim_region  r ON f.id_region  = r.id_region
JOIN dim_produit p ON f.id_produit = p.id_produit
JOIN dim_temps   t ON f.id_date    = t.id_date
GROUP BY r.region_admin, p.categorie, t.trimestre;
```

Avantages ROLAP :
- Capacité à gérer de **très gros volumes** (pas de limite de taille du cube)
- Données toujours **à jour** — pas de recalcul périodique du cube

- Réutilise l'infrastructure SQL existante — pas d'outil supplémentaire
- Flexibilité totale des requêtes ad hoc

Inconvénients ROLAP :
- Performances moindres que MOLAP sur des requêtes répétitives
- Dépend fortement de la qualité de l'indexation et des vues matérialisées

Outils ROLAP : Microsoft Analysis Services (mode ROLAP), MicroStrategy, Metabase, Power BI sur PostgreSQL

MOLAP — Multidimensional OLAP

> **MOLAP** stocke les données dans une **structure de cube physique dédiée**, précalculée et optimisée pour la lecture analytique. Les agrégats sont calculés à l'avance et stockés dans des tableaux multidimensionnels propriétaires.

```
ETL → Calcul des agrégats → Cube MOLAP physique → Requêtes ultra-rapides
```

Fonctionnement :

```
Cube MOLAP Mexora (structure physique précalculée) :
┌──────────────┬──────────────┬──────────────┬──────────────┐
│              │ T1 2024      │ T2 2024      │ T3 2024      │
├──────────────┼──────────────┼──────────────┼──────────────┤
│ Tanger       │ 2.1M MAD     │ 2.4M MAD     │ 1.9M MAD     │
│   Électro    │   850K       │   920K       │   780K       │
│   Mode       │   720K       │   890K       │   650K       │
│   Aliment.   │   530K       │   590K       │   470K       │
├──────────────┼──────────────┼──────────────┼──────────────┤
│ Casablanca   │ 3.5M MAD     │ 3.8M MAD     │ 3.2M MAD     │
│ ...          │ ...          │ ...          │ ...          │
└──────────────┴──────────────┴──────────────┴──────────────┘
← Toutes ces valeurs sont précalculées et stockées physiquement →
```

Avantages MOLAP :
- **Performances exceptionnelles** en lecture — les agrégats sont déjà calculés
- Navigation dans les dimensions quasi-instantanée
- Compression efficace des données dans la structure cube

Inconvénients MOLAP :
- **Limitation de volume** : au-delà d'un certain volume, le cube devient trop lourd
- **Latence** : le cube doit être recalculé périodiquement (souvent la nuit) — les

données ne sont pas en temps réel
- **Explosion combinatoire** : si le cube a beaucoup de dimensions, le nombre de cellules explose (problème de sparsité — beaucoup de cellules vides)
- Dépendance à un outil propriétaire

Outils MOLAP : Microsoft SSAS (mode MOLAP), IBM Cognos TM1, Oracle Essbase

---

HOLAP — Hybrid OLAP

> **HOLAP** est une approche **hybride** qui combine ROLAP et MOLAP : les données détaillées (granularité fine) restent dans le DWH relationnel (ROLAP), tandis que les agrégats précalculés (niveaux élevés de hiérarchie) sont stockés dans une structure cube (MOLAP).

```
Niveau agrégé (Trimestre, Région)    → Structure MOLAP (rapide)
Niveau détaillé (Jour, Ville, SKU)   → Base relationnelle ROLAP (volumineuse)
```

Fonctionnement chez Mexora :

```
Requête : "CA par région et trimestre"
→ HOLAP utilise le cube MOLAP (réponse en millisecondes)

Requête : "Détail des 500 commandes du 15 novembre à Tanger"
→ HOLAP bascule sur PostgreSQL ROLAP (données détaillées)
```

Avantages HOLAP :
- Meilleur des deux mondes : rapidité sur les agrégats + capacité sur le détail
- Scalable sur de gros volumes

Inconvénients HOLAP :
- Architecture plus complexe à maintenir
- Coût de licence élevé pour les outils qui l'implémentent bien

---

Tableau comparatif ROLAP / MOLAP / HOLAP

Critère	ROLAP	MOLAP	HOLAP
Stockage	Base relationnelle	Cube multidimensionnel	Les deux

Critère	ROLAP	MOLAP	HOLAP
Volume de données	Très grand ✅	Limité ⚠️	Grand ✅
Performances agrégats	Moyen	Excellent ✅	Excellent ✅
Données en temps réel	Oui ✅	Non (batch) ⚠️	Partiel
Flexibilité requêtes	Très élevée ✅	Limitée aux hiérarchies	Moyenne
Complexité	Simple	Moyenne	Élevée
Coût	Faible (SQL)	Élevé (licences)	Élevé
Idéal pour	Gros volumes, requêtes ad hoc	Analyses répétitives, rapidité	Volume + performance

> **Recommandation pour Mexora :** Avec PostgreSQL, vous êtes en mode **ROLAP**. C'est le choix correct pour une PME marocaine : flexible, sans coût de licence, et suffisamment performant avec les vues matérialisées et l'indexation appropriée. MOLAP devient pertinent uniquement si les temps de réponse deviennent inacceptables malgré l'optimisation.

4.7 OLAP multidimensionnel — Les opérations fondamentales

Opération	Définition	Exemple Mexora
Roll-up	Monter dans la hiérarchie (agréger)	Jour → Mois → Trimestre → Année
Drill-down	Descendre dans la hiérarchie (détailler)	Région → Province → Ville → Quartier
Slice	Fixer une valeur sur une dimension	"Uniquement les ventes de Tanger"
Dice	Fixer des valeurs sur plusieurs dimensions	"Tanger + Électronique + 2024"
Pivot	Faire pivoter le cube de données	Inverser lignes et colonnes dans un rapport

---

✅ À retenir — Chapitre 4

> **1.** Un Data Warehouse se définit par 4 propriétés d'Inmon : **thématique, intégré, non-volatile, historisé**. Si une propriété manque, ce n'est pas un DWH au sens strict.
>
> **2.** Les **vues matérialisées** sont l'outil le plus puissant pour accélérer les dashboards BI — elles précalculent et stockent physiquement des résultats lourds.
>
> **3.** Le **partitionnement par date** est quasi-systématique en BI : les requêtes portent généralement sur une période limitée, et le partitionnement évite de scanner l'historique complet.
>
> **4.** Les trois approches OLAP ont des compromis distincts : **ROLAP** (SQL relationnel, flexible, gros volumes), **MOLAP** (cube physique, ultra-rapide, volume limité), **HOLAP** (hybride, meilleur des deux). PostgreSQL = ROLAP par nature.
>
> **5.** Les **window functions** (LAG, LEAD, RANK, SUM OVER) et les opérations OLAP (Roll-up, Drill-down, Slice, Dice) sont les outils analytiques SQL les plus puissants — ils remplacent des auto-jointures complexes et définissent comment les analystes naviguent dans les données.

✅ Key takeaways — Chapter 4

> **1.** A Data Warehouse is defined by Inmon's four properties: **subject-oriented, integrated, non-volatile, and time-variant (historized)**. If one is missing, it is not a strict DWH.
>
> **2. Materialized views** are one of the most powerful ways to speed up BI dashboards because they precompute and physically store heavy aggregates.
>
> **3. Date partitioning** is almost systematic in BI. Queries usually target a limited period, and partitioning avoids scanning the full history.
>
> **4.** OLAP approaches have clear trade-offs: **ROLAP** (relational SQL, flexible, very large volumes), **MOLAP** (physical cube, very fast, limited volume), **HOLAP** (hybrid). With PostgreSQL, you are naturally in **ROLAP**.
>
> **5. Window functions** (LAG, LEAD, RANK, SUM OVER) and OLAP operations (roll-up, drill-down, slice, dice) are among the most powerful SQL analytics tools, replacing complex self-joins and enabling multidimensional navigation.

Chapitre 5 — Architectures Modernes de la Donnée | Modern Data Architectures

5.1 Les limites du Data Warehouse classique

Le DWH traditionnel a révolutionné l'analyse. Mais vers 2010, de nouveaux défis ont émergé :

- **Volume** : des téraoctets de logs, IoT, réseaux sociaux — le DWH ne peut pas tout absorber
- **Variété** : images, vidéos, textes libres — pas uniquement du relationnel structuré
- **Vélocité** : besoin d'analyser en temps réel, pas en batch nocturne
- **Flexibilité** : les schémas évoluent vite, le DWH est trop rigide (schéma-before)

Ces contraintes ont généré de nouvelles architectures.

5.2 Le Data Lake

> Un **Data Lake** est un référentiel de stockage centralisé qui stocke les données **dans leur format natif**, sans schéma prédéfini, qu'elles soient structurées, semi-structurées ou non structurées.

> ⚠️ **Attention — le Data Swamp :** Un Data Lake sans gouvernance et sans organisation en zones devient un **Data Swamp** (marécage) : impossible à exploiter, personne ne sait ce qu'il contient.

Organisation en zones Bronze / Silver / Gold

```
ZONE BRONZE (Raw — Données brutes)
├── Données telles que reçues des sources
├── Jamais modifiées — archive fidèle et traçable
├── Format natif (JSON, CSV, images, logs)
└── Mexora : exports MySQL bruts, logs JSON, fichiers livreurs

ZONE SILVER (Cleaned — Données nettoyées)
├── Données nettoyées, déduplicées, standardisées
├── Schéma appliqué, types corrects, villes harmonisées
├── Format Parquet (compressé, performant)
└── Mexora : commandes propres, clients dédupliqués
```

```
ZONE GOLD (Curated — Données prêtes)
├── Données agrégées, modélisées pour l'analyse
├── Équivalent du Data Warehouse — prêtes à consommer
└── Mexora : faits ventes, dimensions, KPIs précalculés
```

DWH vs Data Lake

Critère	Data Warehouse	Data Lake
Schéma	Défini avant chargement (schema-on-write)	Défini à la lecture (schema-on-read)
Types de données	Structurées uniquement	Tous types
Utilisateurs	Analystes BI	Data Scientists, Engineers
Coût stockage	Élevé	Très faible (stockage objet cloud)
Qualité données	Garantie (ETL strict)	Variable (responsabilité utilisateur)
Flexibilité	Faible (schéma rigide)	Très élevée

5.3 Le Data Lakehouse — La convergence

> Le **Data Lakehouse** combine les avantages du Data Lake (stockage économique, flexibilité) et du Data Warehouse (qualité, performances analytiques, transactions ACID).

```
Stockage objet économique (S3 / GCS)
         +
Couche transactionnelle (Delta Lake / Apache Iceberg)
         +
Moteur SQL analytique (Spark / DuckDB / BigQuery)
         =
Data Lakehouse
```

Technologies clés : Delta Lake (Databricks), Apache Iceberg, Apache Hudi

Exemple Mexora futur : Stocker tous les fichiers bruts sur Google Cloud Storage (économique), appliquer Apache Iceberg pour garantir la cohérence transactionnelle, et interroger avec BigQuery pour la performance analytique.

5.4 Lambda Architecture vs Kappa Architecture

Lambda Architecture

> Deux chemins parallèles : un **batch layer** pour l'historique précis, un **speed layer** pour le temps réel approximatif. Les résultats sont fusionnés dans une **serving layer**.

```
Sources → Batch Layer (Spark)  ──────→ Serving Layer → Dashboards Mexora
       ↘ Speed Layer (Kafka)  ────────↗
```

Problème : Deux systèmes parallèles avec deux bases de code différentes — coûteux en maintenance et source d'incohérences.

Kappa Architecture

> Un seul chemin de traitement en **streaming**, capable de rejouer l'historique si nécessaire. Simplifie Lambda en unifiant batch et streaming.

```
Sources → Stream Processing (Kafka + Flink) → Serving Layer → Dashboards Mexora
```

Exemple Mexora temps réel : Détecter une rupture de stock à Tanger en temps réel et déclencher automatiquement une alerte au gestionnaire — sans attendre le batch nocturne.

5.5 Cloud Data Warehousing

Outil	Éditeur	Particularité
BigQuery	Google	Serverless, paiement à la requête
Snowflake	Snowflake Inc.	Séparation compute/storage
Redshift	Amazon AWS	Intégration écosystème AWS
Synapse Analytics	Microsoft Azure	Intégration Power BI native
DuckDB	Open Source	In-process, analytique locale gratuite

> 💡 **DuckDB pour Mexora :** Gratuit, open source, fonctionne localement sans infrastructure Cloud. Excellente performance analytique sur des volumes de

> PME. C'est le meilleur outil pédagogique et un outil sérieux pour les startups marocaines.

✅ À retenir — Chapitre 5

> **1.** Le Data Warehouse classique atteint ses limites face aux données non structurées et au temps réel — ce n'est pas un défaut, c'est une **spécialisation** pour laquelle il a été conçu.
>
> **2.** Un **Data Lake** sans gouvernance devient un **Data Swamp** inutilisable. L'organisation en zones Bronze/Silver/Gold est indispensable pour garder le contrôle.
>
> **3.** Le **Data Lakehouse** est l'architecture dominante des grandes entreprises en 2024-2025 — il réconcilie le stockage économique du Lake et la qualité analytique du Warehouse.
>
> **4.** La distinction batch vs streaming n'est pas qu'une question technique — c'est une question métier : **à quelle fréquence la décision doit-elle être prise ?**
>
> **5.** Le Cloud ne résout pas les problèmes de qualité des données — il les amplifie à grande échelle si l'ETL est mal conçu.

✅ Key takeaways — Chapter 5

> **1.** The classic Data Warehouse reaches its limits with unstructured data and real-time needs. This is not a flaw, it is a **specialization**.
>
> **2.** A **Data Lake** without governance becomes an unusable **Data Swamp**. The Bronze/Silver/Gold zone organization is essential.
>
> **3.** The **Data Lakehouse** has become the dominant architecture in large companies (2024–2025), combining low-cost lake storage with warehouse-grade analytics and reliability.
>
> **4.** Batch vs streaming is not only technical. It is a business question: **how often must decisions be made?**
>
> **5.** The cloud does not solve data quality problems. With poor ETL practices, it can amplify them at scale.

Chapitre 6 — Visualisation & Restitution | Data Visualization & Delivery

6.1 Principes d'un bon dashboard décisionnel

> Un dashboard n'est pas une collection de graphiques. C'est un **outil de décision**. Un bon dashboard répond à une question métier précise et oriente vers une action concrète.

Les 5 principes fondamentaux

1. Une question = un dashboard
Le dashboard Commercial Mexora répond à "comment se portent nos ventes ?". Le dashboard Logistique répond à "nos livraisons sont-elles fiables ?". Ne pas tout mélanger sur un seul écran.

2. Hiérarchie de l'information
Les KPIs les plus importants en haut à gauche. Le détail en bas et à droite. Le regard lit de gauche à droite, du haut vers le bas.

3. Moins c'est plus
Chaque graphique qui ne répond pas à une question métier claire doit être supprimé. Un dashboard avec 20 graphiques ne répond à aucune question efficacement.

4. Le bon graphique pour la bonne donnée

Type de donnée	Visualisation adaptée
Évolution temporelle	Courbe (Line chart)
Comparaison entre catégories	Barres horizontales
Proportion d'un tout	Donut (jamais de camembert 3D)
Corrélation	Nuage de points
Distribution géographique	Carte choroplèthe
Classement (Top N)	Barres horizontales triées

5. Couleur = signification
Rouge = problème / alerte, Vert = objectif atteint, Orange = vigilance. La couleur doit porter du sens, pas de l'esthétique.

6.2 Dashboard Mexora — Conception type

```
┌──────────────────────────────────────────────────────────────────────┐
│  Mexora — Dashboard Commercial Mensuel                Novembre 2024  │
├──────────────────┬──────────────────┬──────────────────┬──────────────────┤
│  CA du mois      │  Commandes       │  Panier moyen    │  Taux retour     │
│  8.4M MAD        │  16 820          │  499 MAD         │  2.8%            │
│  ↑ +14% vs N-1   │  ↑ +9%           │  ↑ +5%           │  ↓ -0.4% 🟢      │
├──────────────────┴──────────────────┴──────────────────┴──────────────────┤
│  CA par Région — Carte Maroc      │  Top 10 Produits (barres horiz.)  │
│  [Carte choroplèthe]              │  [Barres triées par CA]           │
│  Tanger : 2.1M 🟢                 │  1. iPhone 16 — 890K MAD          │
│  Rabat : 1.8M 🟡                  │  2. Samsung TV — 670K MAD         │
│  Casa : 3.5M 🟢                   │  3. Nike Air Max — 450K MAD       │
├──────────────────────────────────┤                                   │
│  Évolution CA 12 mois             │  4. ...                           │
│  [Courbe avec benchmark N-1]      │                                   │
└──────────────────────────────────┴──────────────────────────────────┘
```

6.3 KPIs Mexora — Définitions et calculs SQL

> ⚠️ **Règle d'or :** Les KPIs doivent être définis avec les métiers AVANT d'être calculés. Un KPI sans définition métier claire est une source de conflits et de méfiance dans les données.

```sql
-- KPI 1 : Chiffre d'Affaires mensuel
SELECT SUM(montant_ttc) AS ca_total
FROM dwh_mexora.fait_ventes
WHERE id_date BETWEEN 20241101 AND 20241130;

-- KPI 2 : Panier moyen (mesure non-additive — recalculer)
SELECT
    SUM(montant_ttc) / COUNT(DISTINCT id_commande) AS panie
r_moyen
FROM dwh_mexora.fait_ventes
WHERE id_date BETWEEN 20241101 AND 20241130;

-- KPI 3 : Taux de rétention client (clients avec 2+ comman
des)
SELECT
    ROUND(
```

```
        COUNT(DISTINCT CASE WHEN nb_commandes >= 2 THEN id_
client END) * 100.0
        / NULLIF(COUNT(DISTINCT id_client), 0),
        2
    ) AS taux_retention_pct
FROM (
    SELECT id_client, COUNT(DISTINCT id_commande) AS nb_com
mandes
    FROM dwh_mexora.fait_ventes
    GROUP BY id_client
) t;

-- KPI 4 : Délai moyen de livraison par région
SELECT
    r.region_admin,
    ROUND(AVG(f.delai_livraison_jours), 1) AS delai_moyen_j
ours
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_region r ON f.id_region = r.id_region
WHERE f.delai_livraison_jours IS NOT NULL
  AND f.id_date BETWEEN 20241101 AND 20241130
GROUP BY r.region_admin
ORDER BY delai_moyen_jours;
```

✅ À retenir — Chapitre 6

> **1.** Un dashboard est un **outil de décision**, pas une collection de graphiques — il doit répondre à une question métier précise et orienter vers une action.
>
> **2.** Chaque type de données a un type de graphique optimal. Utiliser un camembert pour comparer des évolutions temporelles est une faute de communication.
>
> **3.** Les **KPIs** doivent être définis en collaboration avec les métiers avant d'être calculés — un KPI sans définition partagée génère des conflits de chiffres entre départements.
>
> **4.** La **hiérarchie visuelle** doit refléter l'importance métier : ce qui est grand et en haut attire l'œil en premier — les KPIs critiques doivent y être.

> **5.** Un dashboard non utilisé par les décideurs est un projet BI qui a échoué — la perfection technique ne compense pas l'inadéquation avec les besoins réels.

✅ Key takeaways — Chapter 6

> **1.** A dashboard is a **decision-making tool**, not a collection of charts. It must answer a clear business question and guide action.
>
> **2.** Each data type has an optimal visualization. Using a pie chart to compare time trends is a communication mistake.
>
> **3. KPIs** must be defined with business stakeholders before being calculated. A KPI without a shared definition leads to conflicting numbers.
>
> **4. Visual hierarchy** should reflect business importance: what is large and placed at the top is seen first. Critical KPIs should be there.
>
> **5.** A dashboard that is not used by decision-makers is a failed BI project. Technical perfection does not compensate for poor business fit.

Chapitre 7 — Visualisation avec Metabase | From SQL to Dashboard

7.1 Pourquoi un outil de visualisation ?

> Les chapitres précédents ont permis de construire le DWH Mexora sous PostgreSQL et de définir les KPIs en SQL. Metabase est l'outil qui **ferme la boucle** : il transforme une requête SQL en graphique interactif, puis assemble les graphiques en dashboard décisionnel.

Le gap que Metabase comble

```
Chapitre 4 — Modélisation DWH     →  PostgreSQL  ✅
Chapitre 5 — Pipeline ETL         →  Python + psycopg2  ✅
Chapitre 6 — KPIs en SQL          →  PostgreSQL  ✅
Chapitre 7 — Dashboard réel       →  Metabase  ← nous sommes ici
```

Pourquoi Metabase et pas Power BI ?

Critère	Metabase	Power BI
Licence	Open source, gratuit	Microsoft, payant
OS supportés	Windows, Mac, Linux	Windows uniquement
Connexion PostgreSQL	Native, immédiate	Driver ODBC externe requis
Langage de calcul	SQL standard	DAX (propriétaire Microsoft)
Lien SQL → graphique	Direct et visible	Caché derrière l'interface
Courbe d'apprentissage	Faible	Moyenne à élevée

> **Principe pédagogique :** Metabase ne cache pas la logique SQL — l'étudiant voit directement le lien entre sa requête et sa visualisation. C'est l'outil le plus adapté pour apprendre les fondamentaux de la BI avant de passer à des outils enterprise en Master.

7.2 Installation de Metabase

Option A — Java JAR (recommandée en salle TP)

Prérequis : Java 11 ou supérieur installé.

```
# Vérifier la version Java
java -version

# Télécharger Metabase
wget https://downloads.metabase.com/latest/metabase.jar

# Lancer Metabase
java -jar metabase.jar
```

Ouvrir le navigateur sur : **http://localhost:3000**

Option B — Docker

```
docker run -d -p 3000:3000 --name metabase metabase/metabas
e
```

Option C — Metabase Cloud (démo rapide sans installation)

Créer un compte gratuit sur **https://www.metabase.com** — aucune installation, accès immédiat pour 14 jours.

7.3 Connexion à la base dwh_mexora

Étape 1 — Premier lancement

Au premier démarrage, Metabase lance un assistant de configuration :

```
1. Créer un compte administrateur (email + mot de passe)
2. Sélectionner "Add your data"
3. Choisir le type de base : PostgreSQL
```

Étape 2 — Paramètres de connexion

Champ	Valeur
Display name	DWH Mexora
Host	localhost
Port	5432
Database name	mexora_dwh
Username	votre_utilisateur
Password	votre_mot_de_passe

Cliquer sur **"Save"** → Metabase teste la connexion et indexe les schémas.

Étape 3 — Vérification

Une fois connecté, Metabase affiche les schémas disponibles :

```
DWH Mexora
├── staging_mexora
├── dwh_mexora
│   ├── fait_ventes
│   ├── dim_temps
│   ├── dim_produit
│   ├── dim_region
│   └── dim_client
└── reporting_mexora
    └── mv_ca_mensuel
```

7.4 Interface SQL native — Le mode essentiel

> Metabase propose deux modes d'interrogation : le **Question Builder** (no-code, drag & drop) et le **SQL Editor** (requêtes SQL directes). En cours, on utilise **exclusivement le SQL Editor** pour maintenir le lien avec les fondamentaux appris aux chapitres précédents.

Accéder au SQL Editor

```
Menu principal → New → SQL query → Sélectionner "DWH Mexora"
```

Premier test — CA du mois de Novembre 2024

```sql
SELECT SUM(montant_ttc) AS ca_total
FROM dwh_mexora.fait_ventes
WHERE id_date BETWEEN 20241101 AND 20241130;
```

Résultat affiché : un tableau avec une cellule → **8 400 000 MAD**

Pour transformer en **KPI card** :

```
Visualization → Number
```

Metabase affiche le chiffre en grand, formaté, prêt pour un dashboard.

7.5 Les KPIs Mexora dans Metabase

Pour chaque KPI du Chapitre 6, voici le type de visualisation à choisir dans Metabase.

KPI 1 — Chiffre d'Affaires mensuel → Number

```sql
SELECT SUM(montant_ttc) AS ca_total
FROM dwh_mexora.fait_ventes
WHERE id_date BETWEEN 20241101 AND 20241130;
```

Visualization → Number

Configuration :
- Label : “CA Novembre 2024”
- Prefix : aucun
- Suffix : ” MAD”

KPI 2 — Évolution du CA sur 12 mois → Line chart

```sql
SELECT
    t.annee,
    t.mois,
    t.libelle_mois,
    SUM(f.montant_ttc) AS ca_mois
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_temps t ON f.id_date = t.id_date
WHERE t.annee = 2024
GROUP BY t.annee, t.mois, t.libelle_mois
ORDER BY t.mois;
```

Visualization → Line

Configuration :
- Axe X : `libelle_mois`
- Axe Y : `ca_mois`
- Label : “Évolution CA 2024”

KPI 3 — CA par région → Bar chart horizontal

```sql
SELECT
    r.region_admin,
    SUM(f.montant_ttc) AS ca_region
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_region r ON f.id_region = r.id_region
WHERE f.id_date BETWEEN 20241101 AND 20241130
GROUP BY r.region_admin
ORDER BY ca_region DESC;
```

Visualization → Bar

Configuration :
- Orientation : Horizontal
- Axe X : `ca_region`
- Axe Y : `region_admin`
- Trier par valeur décroissante

KPI 4 — Top 10 produits → Bar chart

```
SELECT
    p.nom_produit,
    p.categorie,
    SUM(f.montant_ttc)      AS ca_produit,
    SUM(f.quantite_vendue)  AS volume_vendu
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t
WHERE f.id_date BETWEEN 20241101 AND 20241130
GROUP BY p.nom_produit, p.categorie
ORDER BY ca_produit DESC
LIMIT 10;
```

Visualization → Bar (horizontal, trié)

KPI 5 — Panier moyen → Number

> ⚠️ **Rappel Chapitre 6 :** Le panier moyen est une mesure **non-additive**. Il ne faut jamais faire la moyenne des paniers moyens journaliers — recalculer depuis les données brutes.

```
SELECT
    ROUND(
        SUM(montant_ttc) / NULLIF(COUNT(DISTINCT id_command
e), 0),
        2
    ) AS panier_moyen
FROM dwh_mexora.fait_ventes
WHERE id_date BETWEEN 20241101 AND 20241130;
```

Visualization → **Number**, suffix " MAD"

KPI 6 — Répartition par catégorie → Donut

```
SELECT
    p.categorie,
    SUM(f.montant_ttc) AS ca_categorie
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t
WHERE f.id_date BETWEEN 20241101 AND 20241130
GROUP BY p.categorie
ORDER BY ca_categorie DESC;
```

Visualization → **Pie** (Metabase appelle le Donut "Pie" dans son interface)

> ⚠️ **Note pédagogique :** Le donut/camembert est acceptable pour 3 à 5 catégories maximum. Au-delà, passer aux barres horizontales triées — l'œil compare mieux des longueurs que des angles.

7.6 Construction du Dashboard Mexora

Étape 1 — Sauvegarder chaque question

Pour chaque requête créée en 7.5 :

```
Run → Save → Nommer la question (ex: "KPI - CA Novembre 2024")
→ Collection : "Mexora Commercial"
```

Étape 2 — Créer le dashboard

```
New → Dashboard → Nom : "Dashboard Commercial Mexora – Novembre 2024"
```

Étape 3 — Ajouter les questions au dashboard

```
+ Add a question → Sélectionner depuis la collection "Mexora Commercial"
```

Disposition recommandée (cohérente avec le Chapitre 6) :

```
┌──────────────────────────────────────────────────────────────┐
│  Dashboard Commercial Mexora              Novembre 2024      │
├────────────────┬──────────────────┬────────────────┬──────────┤
│  CA du mois    │  Nb commandes    │ Panier moyen   │ Top région │
│  [Number]      │  [Number]        │  [Number]      │ [Number]   │
├────────────────┴──────────────────┴────────────────┴──────────┤
│  Évolution CA 12 mois [Line chart]                            │
├──────────────────────────────────┬────────────────────────────┤
│  CA par région [Bar horizontal]  │  Top 10 produits [Bar]     │
├──────────────────────────────────┴────────────────────────────┤
│  Répartition par catégorie [Donut]                            │
└──────────────────────────────────────────────────────────────┘
```

Étape 4 — Ajouter un filtre temporel interactif

```
Dashboard → Edit → Add a filter → Time → "Date range"
```

Lier ce filtre aux questions qui contiennent `id_date` :

```
Filter mapping → fait_ventes.id_date
```

L'analyste peut maintenant filtrer tout le dashboard sur n'importe quelle période sans modifier le SQL.

Étape 5 — Sauvegarder et partager

```
Save → Done editing
```

Pour partager en lecture seule :

```
Share → Public link (génère une URL sans authentification)
```

7.7 Vues matérialisées et performances dans Metabase

> Lorsque les requêtes deviennent lentes sur de grands volumes, on utilise la vue matérialisée `reporting_mexora.mv_ca_mensuel` créée au Chapitre 4 — au lieu de requêter directement `fait_ventes`.

Remplacer une requête lente par la vue matérialisée

Avant (lent sur gros volumes) :

```
SELECT t.annee, t.mois, r.region_admin, SUM(f.montant_ttc)
AS ca
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_temps t   ON f.id_date   = t.id_date
JOIN dwh_mexora.dim_region r  ON f.id_region = r.id_region
GROUP BY t.annee, t.mois, r.region_admin;
```

Après (lecture directe de la vue précalculée) :

```
SELECT annee, mois, region_admin, ca_ttc
FROM reporting_mexora.mv_ca_mensuel
ORDER BY annee, mois;
```

Le résultat est identique, mais la requête est **quasi-instantanée** car les agrégats sont déjà calculés.

> **Règle pratique :** Dans Metabase, si un graphique met plus de 3 secondes à se charger, envisager une vue matérialisée dans PostgreSQL. Le problème de performance se règle **dans la base de données**, pas dans l'outil de visualisation.

✅ À retenir — Chapitre 7

> **1.** Metabase est l'outil qui **ferme la boucle** entre les requêtes SQL (Chapitre 6) et les dashboards décisionnels. Sans outil de visualisation, un cours de BI reste incomplet.
>
> **2.** Le **SQL Editor de Metabase** est le mode à privilégier en formation — il maintient le lien direct entre la requête et la visualisation, et transfère sur n'importe quel autre outil BI.

> **3.** Chaque type de KPI a un type de visualisation optimal dans Metabase : **Number** pour les indicateurs ponctuels, **Line** pour les évolutions temporelles, **Bar horizontal** pour les classements, **Pie/Donut** pour les proportions (≤ 5 catégories).
>
> **4.** Le **filtre temporel interactif** du dashboard permet à l'analyste de naviguer dans les périodes sans modifier le SQL — c'est l'équivalent d'une opération **Slice** sur le cube OLAP (Chapitre 4).
>
> **5.** Les problèmes de **performance** se règlent dans PostgreSQL (vues matérialisées, indexation) — pas dans Metabase. Un dashboard lent est un signal d'optimisation côté base de données.

✅ Key takeaways — Chapter 7

> **1.** Metabase **closes the loop** between SQL queries (Chapter 6) and actual BI dashboards. Without a visualization tool, a BI course remains theoretical.
>
> **2.** The **SQL Editor** in Metabase is the mode to use in training — it keeps the direct link between query and visualization, and the skills transfer to any other BI tool.
>
> **3.** Each KPI type has an optimal visualization: **Number** for point-in-time indicators, **Line** for time trends, **Horizontal Bar** for rankings, **Pie/Donut** for proportions (≤ 5 categories).
>
> **4.** The **interactive time filter** lets analysts navigate across periods without modifying SQL — this is the dashboard equivalent of a **Slice** operation on the OLAP cube (Chapter 4).
>
> **5. Performance problems** are solved in PostgreSQL (materialized views, indexes) — not in Metabase. A slow dashboard is a signal to optimize the database layer.

Chapitre 8 — Gouvernance & Qualité des Données | Data Governance & Data Quality

8.1 Data Quality — Les 6 dimensions

> La **qualité des données** mesure dans quelle mesure les données sont adaptées à l'usage qu'on souhaite en faire. Des données de mauvaise qualité génèrent des décisions incorrectes — parfois plus dangereuses que l'absence de données.

Dimension	Définition	Exemple Mexora
Complétude	Les données requises sont-elles présentes ?	7% des commandes sans région associée
Exactitude	Les données reflètent-elles la réalité ?	Prix en base ≠ prix réel facturé au client
Cohérence	Les mêmes données sont-elles identiques dans toutes les sources ?	"Tanger" vs "TNG" vs "tanger"
Fraîcheur	Les données sont-elles à jour ?	Stock J-1 alors que la direction veut le stock temps réel
Unicité	Y a-t-il des doublons ?	Même client enregistré 3 fois avec emails différents
Validité	Les données respectent-elles les règles métier ?	Quantité commandée = -5 (impossible)

8.2 Data Governance — Définition et rôles

> La **gouvernance des données** est l'ensemble des politiques, processus, rôles et responsabilités qui garantissent que les données de Mexora sont **fiables, sécurisées, accessibles** et utilisées de manière conforme.

> ⚠️ **Point critique :** La gouvernance n'est pas un projet technique. C'est un projet **organisationnel**. Sans engagement des directions métier, aucune gouvernance technique ne fonctionne durablement.

Rôles clés en gouvernance

Data Owner (Propriétaire métier)
Responsable d'un domaine de données. Décide qui peut accéder à quoi. Chez Mexora : le Directeur Commercial est Data Owner des données ventes.

Data Steward (Gardien de la qualité)
Garant de la qualité des données dans son domaine. Définit les règles de nettoyage et de validation.

Data Engineer
Implémente techniquement les règles de gouvernance dans les pipelines. Exécute les règles définies par le Data Steward.

8.3 RGPD et données personnelles chez Mexora

> Le **RGPD** (Règlement Général sur la Protection des Données) s'applique à toutes les entreprises traitant des données de résidents européens. Au Maroc, la **Loi 09-08** régit la protection des données personnelles avec des principes similaires.

Données sensibles chez Mexora

Les données clients (nom, email, adresse, téléphone, historique d'achats) sont des **données personnelles** soumises à la Loi 09-08. Elles doivent être protégées dans le DWH.

Pseudonymisation — concilier analyse et vie privée

```
import hashlib

def pseudonymiser_client(email: str, nom: str, prenom: str)
-> dict:
    """
    Remplace les identifiants directs par des hash irrévers
ibles.
    Conserve les attributs analytiques non-identifiants.
    """
    return {
        # Identifiants pseudonymisés (hash SHA-256)
        'email_hash': hashlib.sha256(email.lower().encode
()).hexdigest(),
        'nom_hash':   hashlib.sha256(f"{nom}{prenom}".lower
().encode()).hexdigest(),

        # Attributs analytiques conservés — non-identifiant
s
        'ville':          ville,
        'region':         region,
```

```
        'segment_client': segment,
        'tranche_age':    calculer_tranche_age(age),  # ex: "25-34"
        'sexe':           sexe
    }
```

Droit à l'effacement dans un DWH

Le DWH est non-volatile par définition — mais le RGPD et la Loi 09-08 imposent le droit à l'oubli. La solution : **pseudonymiser** plutôt que supprimer les données personnelles. On anonymise les champs identifiants tout en conservant les données analytiques agrégées.

8.4 Data Lineage — La traçabilité

> Le **Data Lineage** (lignage des données) décrit le chemin complet d'une donnée depuis sa source jusqu'à sa consommation finale. Il répond à la question : *"D'où vient ce chiffre et comment a-t-il été calculé ?"*

Exemple Mexora :

```
MySQL.commandes.montant_ht (source)
    → ETL: filtre (statut='livré'), conversion format date
    → staging_mexora.commandes_clean.montant_ht
    → dwh_mexora.fait_ventes.montant_ht
    → reporting_mexora.mv_ca_mensuel.ca_ht
    → Dashboard Power BI : "CA mensuel par région"
    → Rapport CODIR : "Performance commerciale novembre 2024"
```

Quand le Directeur dit "ce chiffre est faux", le lineage permet de localiser l'anomalie en quelques minutes plutôt qu'en plusieurs jours d'investigation.

✅ À retenir — Chapitre 8

> **1.** La qualité des données se mesure selon 6 dimensions : complétude, exactitude, cohérence, fraîcheur, unicité, validité. Un projet BI doit mesurer et améliorer **toutes** ces dimensions.
>
> **2. "Garbage in, Garbage out"** est la loi fondamentale. Des décisions prises sur des données de mauvaise qualité sont souvent **pires** que des décisions

> prises sans données — la fausse certitude est plus dangereuse que l'incertitude avouée.
>
> **3.** La gouvernance des données est un **projet organisationnel**, pas technique. Sans engagement des Data Owners métier, aucune règle technique ne sera respectée durablement.
>
> **4.** La **pseudonymisation** est la technique recommandée pour concilier analyse BI et respect de la Loi 09-08 (Maroc) / RGPD (Europe) — on anonymise les identifiants, on conserve les attributs analytiques.
>
> **5.** Le **Data Lineage** n'est pas un luxe — c'est l'outil qui permet de **faire confiance aux chiffres** et de diagnostiquer rapidement toute anomalie dans la chaîne de données.

✅ Key takeaways — Chapter 8

> **1.** Data quality is measured across six dimensions: completeness, accuracy, consistency, timeliness (freshness), uniqueness, and validity. A BI project must measure and improve **all** of them.
>
> **2. Garbage in, garbage out** is the fundamental law. Decisions based on poor-quality data are often **worse** than decisions made without data, because false certainty is dangerous.
>
> **3.** Data governance is an **organizational** project, not only a technical one. Without business Data Owner commitment, technical rules will not last.
>
> **4. Pseudonymization** is recommended to reconcile BI analytics with privacy regulations (Morocco Law 09-08 / GDPR): anonymize direct identifiers while keeping analytical attributes.
>
> **5. Data lineage** is not a luxury. It is what allows you to **trust the numbers** and quickly diagnose anomalies across the data pipeline.

Chapitre 9 — Perspectives : IA & Futur de la Donnée | AI & the Future of Data

9.1 L'IA transforme chaque couche du pipeline

L'intelligence artificielle ne remplace pas l'ingénierie de données — elle s'appuie sur elle et l'amplifie. Chaque étape du pipeline BI est impactée :

```
SOURCES        → IA détecte automatiquement les schémas, anomalies et dérives
ETL            → LLMs génèrent du code de transformation, suggèrent des règles de nett
oyage
DATA WAREHOUSE → IA optimise les requêtes et l'indexation automatiquement
RESTITUTION    → IA génère des insights en langage naturel, crée des dashboards auto
DÉCISION       → IA prédit, recommande, automatise certaines décisions répétitives
```

> ⚠️ **Mise en garde fondamentale :** Un LLM qui génère du SQL sur un **mauvais modèle dimensionnel** génère du SQL parfaitement incorrect — mais avec une confiance apparente. La qualité de l'architecture de données reste la responsabilité du Data Engineer, pas de l'IA.

9.2 BI Augmentée — Interroger les données en langage naturel

> La **BI augmentée** permet à des utilisateurs non-techniques d'interroger un Data Warehouse en **langage naturel**, sans écrire de SQL. C'est la démocratisation de l'accès aux données.

Exemple Mexora avec Text-to-SQL :

```
Directeur Commercial → "Quels sont nos 5 meilleurs clients en CA
                         depuis le début de l'année dans la région Tanger-Tétouan ?"

Système IA → génère et exécute automatiquement :

SELECT
    c.nom_complet,
    SUM(f.montant_ttc) AS ca_total
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_client c ON f.id_client  = c.id_client
JOIN dwh_mexora.dim_region r ON f.id_region  = r.id_region
JOIN dwh_mexora.dim_temps  t ON f.id_date    = t.id_date
WHERE r.region_admin ILIKE '%Tanger%'
  AND t.annee = 2024
GROUP BY c.nom_complet
ORDER BY ca_total DESC
LIMIT 5;
```

Outils actuels : Amazon Q, Google Looker Studio AI, Microsoft Copilot in Power BI, Metabase AI

9.3 DataOps — La BI comme produit logiciel

> **DataOps** applique les principes du DevOps (intégration continue, tests automatisés, déploiement continu) aux pipelines de données.

Ce que ça change chez Mexora :
- Les pipelines ETL sont versionnés dans Git — comme du code applicatif
- Chaque modification passe par des tests automatisés avant déploiement
- Des alertes automatiques détectent les anomalies de données en production (ex : le CA du jour est 0 — est-ce réel ou un bug ETL ?)

Exemple de tests automatiques avec dbt :

```
# Tests de qualité automatiques sur le DWH Mexora
models:
-name: fait_ventes
tests:
-not_null:
columns:[id_date, id_produit, id_client, montant_ttc]
-relationships:
to: ref('dim_produit')
field: id_produit
-accepted_values:
column_name: statut_commande
values:['livré','annulé','en_cours','retourné']
-expression_tests:
- montant_ttc > 0
- quantite_vendue > 0
```

9.4 Data Mesh — La décentralisation

> Le **Data Mesh** traite les données comme des **produits**, gérés de manière **décentralisée** par les équipes métier qui les produisent — plutôt que par une équipe data centrale qui devient un goulot d'étranglement.

Avec le Data Mesh chez Mexora (à grande échelle) :
- L'équipe Ventes gère et expose ses propres données comme un produit consommable
- L'équipe Logistique gère les siennes de façon autonome
- Une plateforme commune (Data Platform) garantit l'interopérabilité

> **Note :** Le Data Mesh est pertinent pour les grandes organisations (50+ personnes data). Pour Mexora aujourd'hui, c'est prématuré. Comprendre le concept est important pour anticiper l'évolution future.

9.5 Les compétences de demain

Compétence	Importance 2025	Tendance
SQL avancé	⭐⭐⭐⭐⭐	Stable — fondamental indépassable
Python / pandas	⭐⭐⭐⭐⭐	Stable — incontournable
Modélisation dimensionnelle	⭐⭐⭐⭐⭐	En hausse — plus valorisée qu'avant
Cloud (GCP / AWS / Azure)	⭐⭐⭐⭐	Forte croissance
dbt (data build tool)	⭐⭐⭐⭐	Très forte croissance
Apache Spark	⭐⭐⭐⭐	Stable sur les gros volumes
Streaming (Kafka)	⭐⭐⭐	En croissance
DataOps / CI-CD data	⭐⭐⭐	En forte croissance
Prompt Engineering	⭐⭐⭐	En croissance
Data Governance	⭐⭐⭐⭐	Très forte croissance (RGPD, Loi 09-08)

> **La compétence la plus rare et la plus valorisée** n'est pas technique : c'est la capacité à **traduire un besoin métier en architecture de données**. C'est précisément ce que ce cours vous prépare à faire.

✅ À retenir — Chapitre 9

> **1.** L'IA générative amplifie les compétences d'un bon Data Engineer — elle ne les remplace pas. Un mauvais modèle de données produit de mauvaises réponses IA, quelle que soit la puissance du LLM.

> **2.** La **BI augmentée** (Text-to-SQL, insights automatiques) démocratise l'accès aux données. Mais la définition des KPIs et la validation des résultats restent des responsabilités humaines.
>
> **3. DataOps** n'est pas optionnel en 2025 — les pipelines data doivent être testés, versionnés et monitorés avec la même rigueur que du code de production.
>
> **4.** Le **SQL restera fondamental** pour les 10 prochaines années — les LLMs génèrent du SQL, mais il faut savoir lire, comprendre et valider ce qu'ils génèrent.
>
> **5.** Les compétences les plus précieuses combinent technique et métier : **architecturer des données pour répondre à des besoins décisionnels réels** — c'est l'essence du métier de Data Engineer.

✅ Key takeaways — Chapter 9

> **1.** Generative AI amplifies a strong Data Engineer's skills, it does not replace them. With a poor data model, AI produces poor answers,
>
> **2. Augmented BI** (text-to-SQL, automated insights) democratizes data access. But KPI definitions and result validation remain human responsibilities.
>
> **3. DataOps** is not optional in 2025. Data pipelines must be tested, versioned, and monitored with the same rigor as production software.
>
> **4. SQL will remain fundamental** for the next decade. LLMs can generate SQL, but you must be able to read, understand, and validate it.
>
> **5.** The most valuable skills combine tech and business: **designing data architectures that answer real decision-making needs**. That is the core of the Data Engineer role.

📚 Références et ressources

Livres de référence

- **The Data Warehouse Toolkit** — Ralph Kimball & Margy Ross *(bible de la modélisation dimensionnelle)*
- **Building the Data Warehouse** — W.H. Inmon *(approche top-down, définitions fondatrices)*

- **Fundamentals of Data Engineering** — Joe Reis & Matt Housley, 2022 *(vision moderne)*
- **Designing Data-Intensive Applications** — Martin Kleppmann *(architectures avancées)*

Stack technique recommandé

Rôle	Outil	Justification
DWH	PostgreSQL	Open source, puissant, idéal PME marocaine
ETL	Python + pandas + SQLAlchemy	Standard du marché, flexible
Analytique locale	DuckDB	Gratuit, performant, SQL natif
Transformations SQL	dbt	Standard DataOps, versionnable
Visualisation	Metabase	Open source, déployable on-premise
Visualisation entreprise	Power BI Desktop	Standard Maroc et international

Ressources en ligne

- Documentation PostgreSQL : postgresql.org/docs
- dbt Learn (gratuit) : courses.getdbt.com
- DuckDB documentation : duckdb.org/docs

Licence Analytique des Données, Année universitaire 2025-2026

📓 Miniprojet 1 — Pipeline ETL & Data Warehouse

💼 Miniprojet 2 — Data Lake & Analyse du Marché de l'Emploi IT au Maroc

📝 Série d'Exercices — Ingénierie de Données & Business Intelligence